# Searching Algorithms and Complexity

## Linear search, binary search, and why sorted data matters

CSCI 235 – Data Structures

Hunter College, Fall 2026

Class 19 (continued)  Thursday, November 12, 2026

# Today

1. The **guessing game**: how fast can you find a number?
2. **Linear search**: look at everything. Cost $O(n)$.
3. **Binary search**: throw away half each time. Cost $O(\log n)$. It needs **sorted** data.
4. A trace with `lo`, `hi`, `mid`, and the bugs that usually hide there.

- **Exercise 8 (Shunting-Yard Trace)** is due **today, Thursday, Nov 12**.
- Sorting starts next class (Tuesday, Nov 17). Today's ideas on $O(\log n)$ and "halving" return there.

# Question: Guess My Number

I am thinking of a whole number from **1 to 100**. After each guess I say **higher** or **lower**.

**(a)** What is a good first guess? Why?
**(b)** What is the **largest** number of guesses you could ever need with your strategy?

# Question: Guess My Number

I am thinking of a whole number from **1 to 100**. After each guess I say **higher** or **lower**.

**(a)** What is a good first guess? Why?
**(b)** What is the **largest** number of guesses you could ever need with your strategy?

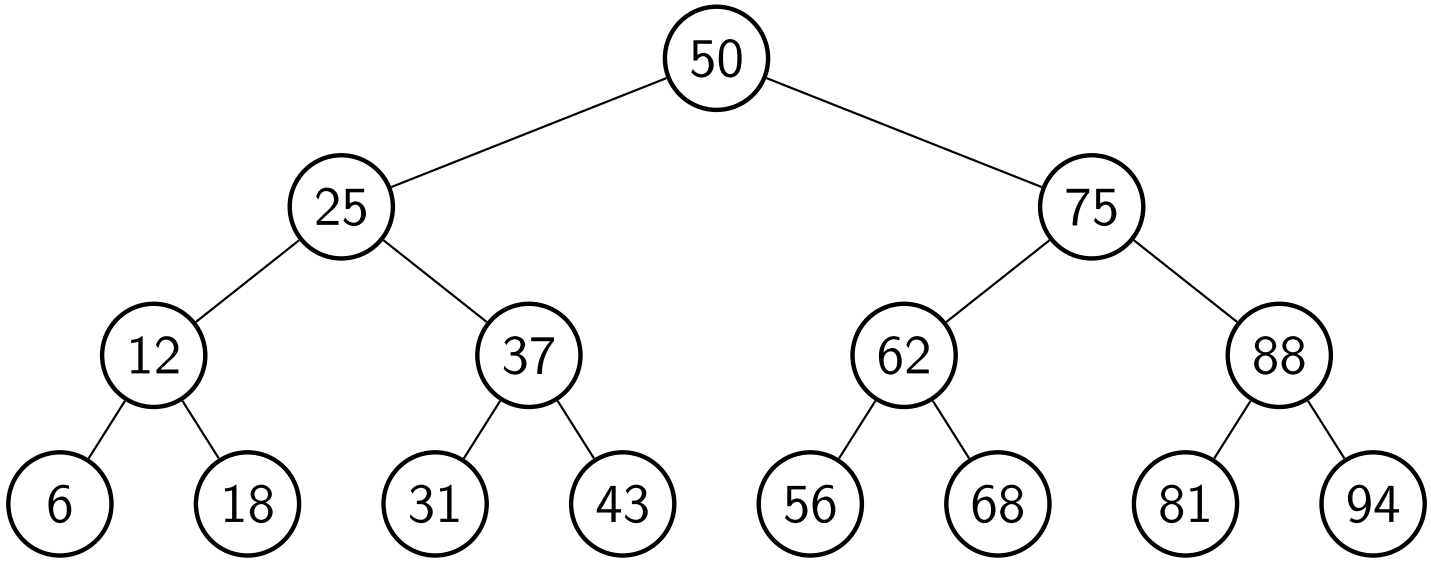
**(a) 50**, the middle: whatever I say, you remove **half** of the possibilities.

**(b) 7.** Each guess halves the range:
$100 \to 50 \to 25 \to 12 \to 6 \to 3 \to 1$. In general you need $\lfloor \log_2 n \rfloor + 1$ guesses. Draw the guesses as a tree: $g$ levels of guesses hold at most $2^g - 1$ numbers, that is $2^6 - 1 = 63 < 100$ for six guesses, but $2^7 - 1 = 127 \geq 100$ for seven.

**Analogy:** looking up a word in a paper dictionary. You open it near the middle and decide "before or after?" You never read it page by page. That only works because the words are **in order**.

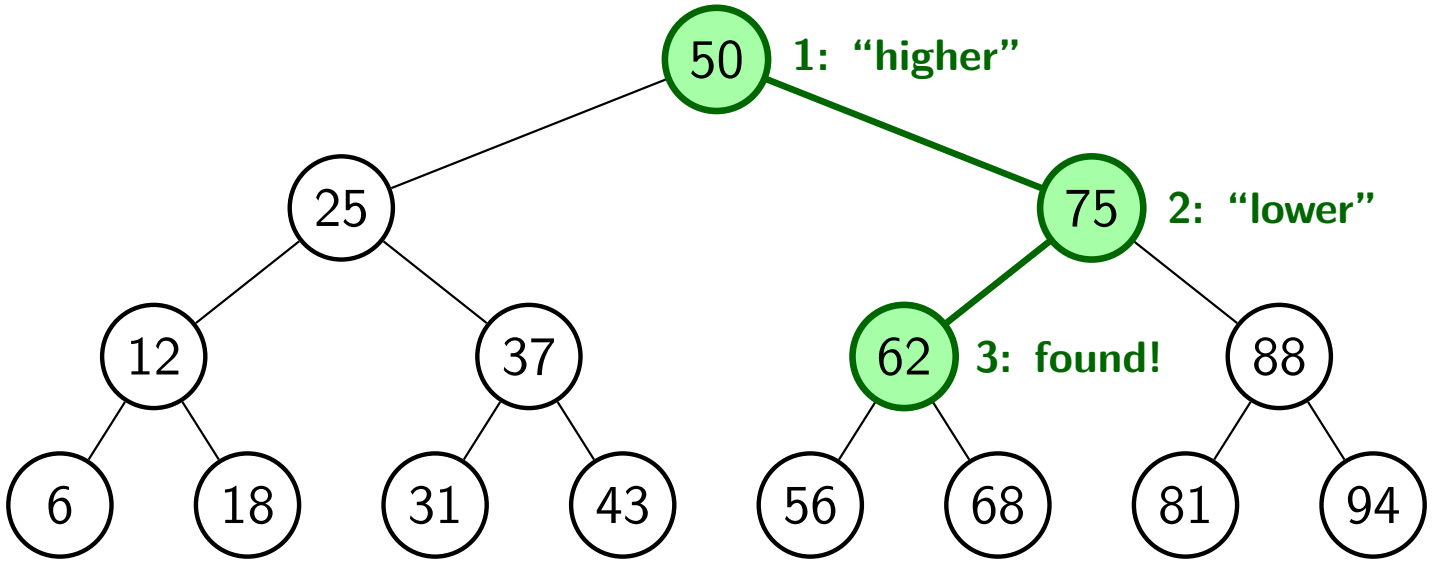
# The Guessing Game as a Tree

Draw every guess as a **node** (numbered 1, 2, 3 below in the order guessed). After "**higher**" go to the **right** child, after "**lower**" go to the **left** child. First guess 50 is the **root**; the middles 25 and 75 are its children.



# The Guessing Game as a Tree

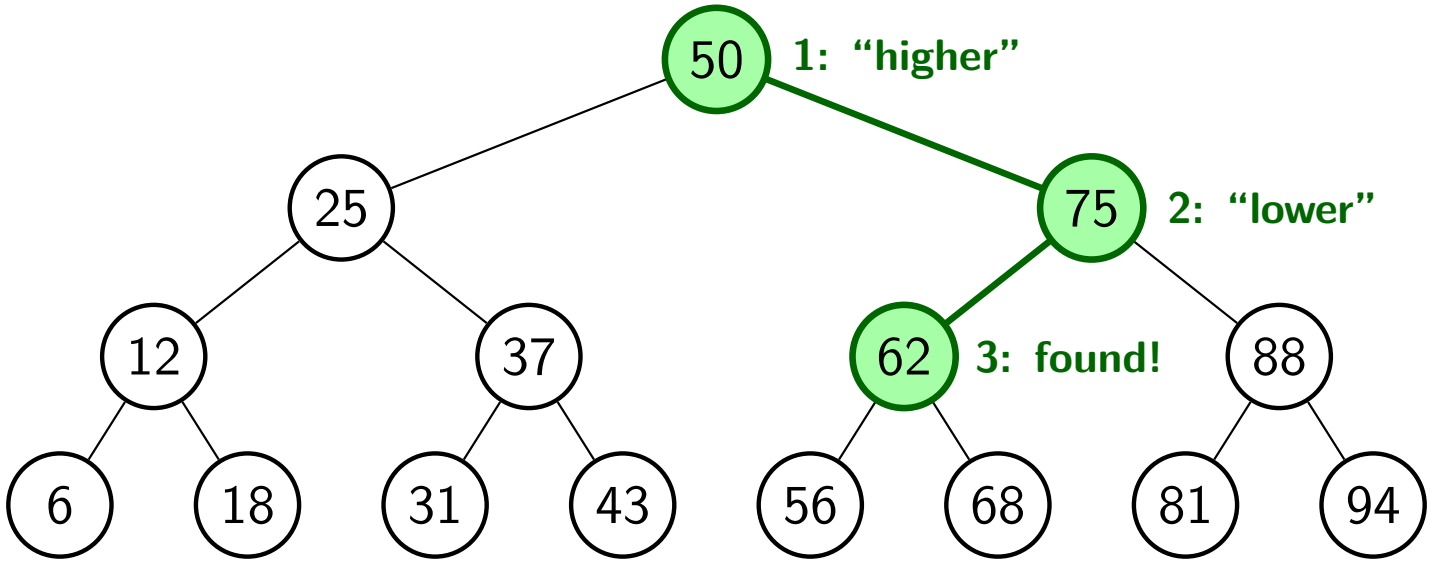
Draw every guess as a **node** (numbered 1, 2, 3 below in the order guessed). After "**higher**" go to the **right** child, after "**lower**" go to the **left** child. First guess 50 is the **root**; the middles 25 and 75 are its children.




**Secret number 62:** guess 50 (higher), guess 75 (lower), guess 62: found in 3 guesses.

# The Guessing Game as a Tree

Draw every guess as a **node** (numbered 1, 2, 3 below in the order guessed). After "**higher**" go to the **right** child, after "**lower**" go to the **left** child. First guess 50 is the **root**; the middles 25 and 75 are its children.




**Secret number 62:** guess 50 (higher), guess 75 (lower), guess 62: found in 3 guesses. Every smaller number is in a node's **left** subtree, every larger one in its **right** subtree: a **binary search tree**. The tree has 7 levels in all (only 4 are drawn), so you need **at most 7 guesses**.

# Linear Search: What Should It Do?

**Specification.** Given a vector `a` and a `target`, **return the index of the first appearance** of `target` in `a` if it is found; otherwise **return** $-1$.

| call | returns | why |
|---|---|---|
| `a = {7, 3, 9, 4, 9}`, `target = 9` | 2 | |
| `a = {7, 3, 9, 4, 9}`, `target = 5` | $-1$ | |
| `a = {}` (empty), `target = 5` | $-1$ | |

# Linear Search: What Should It Do?

**Specification.** Given a vector `a` and a `target`, **return the index of the first appearance** of `target` in `a` if it is found; otherwise **return** $-1$.

| **call** | **returns** | **why** |
|---|---|---|
| `a = {7, 3, 9, 4, 9}`, `target = 9` | 2 | first 9 is at index 2 (not 4) |
| `a = {7, 3, 9, 4, 9}`, `target = 5` | $-1$ | 5 does not appear |
| `a = {}` (empty), `target = 5` | $-1$ | nothing to look at |

# Linear Search: What Should It Do?

**Specification.** Given a vector `a` and a `target`, **return the index of the first appearance** of `target` in `a` if it is found; otherwise **return** $-1$.

| **call** | **returns** | **why** |
|---|---|---|
| `a = {7, 3, 9, 4, 9}`, `target = 9` | 2 | first 9 is at index 2 (not 4) |
| `a = {7, 3, 9, 4, 9}`, `target = 5` | $-1$ | 5 does not appear |
| `a = {}` (empty), `target = 5` | $-1$ | nothing to look at |

**Why $-1$?** Valid indices are $0, 1, 2, \ldots$, so $-1$ can never be confused with a real position.

# Your Turn: Choose the Function Header

We want to **return an index** and **look in a vector for a target**. Which header fits the specification?

A `void linearSearch(std::vector<int> a, int target)`

B `int linearSearch(const std::vector<int>& a, int target)`

C `bool linearSearch(const std::vector<int>& a, int target)`

D `int linearSearch(const std::vector<int>& a)`

# Your Turn: Choose the Function Header

We want to **return an index** and **look in a vector for a target**. Which header fits the specification?

A
```
void linearSearch(std::vector<int> a, int target)
```

B
```
int linearSearch(const std::vector<int>& a, int target)
```

C
```
bool linearSearch(const std::vector<int>& a, int target)
```

D
```
int linearSearch(const std::vector<int>& a)
```

**Raise your hand for A, B, C, or D.**

# Your Turn: Choose the Function Header

We want to **return an index** and **look in a vector for a target**. Which header fits the specification?

A `void linearSearch(std::vector<int> a, int target)`

B `int linearSearch(const std::vector<int>& a, int target)`

C `bool linearSearch(const std::vector<int>& a, int target)`

D `int linearSearch(const std::vector<int>& a)`

**Answer: B.**

- **A:** `void` returns nothing, so no index can come back. It also copies the whole vector.
- **C:** `bool` can only say "found" or "not found", not *where*.
- **D:** there is no `target`: what would we look for?
- **B:** `int` for the index; `const std::vector<int>&` passes the vector **without copying** and promises not to change it; `target` is a small value, passed by value.

# Building It: Header, Then the Loop

**Step 1: the header.** The body is empty for now.

```
int linearSearch(const std::vector<int>& a, int target) {
    // ... to be written
}
```

# Building It: Header, Then the Loop

**Step 2: the repetition statement.** We must look at **every index** from the front, until we find the target.

```
int linearSearch(const std::vector<int>& a, int target) {
    for (int i = 0; i < static_cast<int>(a.size()); ++i) {
        // look at a[i]
    }
}
```

`static_cast<int>(a.size())`: `size()` is an *unsigned* number; the cast lets us compare it with the `int i` without a warning.

# Building It: Header, Then the Loop

**Step 3: the test inside the loop.** If the current element is the target, we are done: **stop at once**. That is why we get the **first** appearance.

```
int linearSearch(const std::vector<int>& a, int target) {
    for (int i = 0; i < static_cast<int>(a.size()); ++i) {
        if (a[i] == target) { return i; } // found: stop at once
    }
}
```

**Is it finished?** Not yet: what happens if the loop ends without ever finding the target? (Next question.)

# Your Turn: Choose the Loop

The loop must visit **every index exactly once**, for any vector including an empty one. Which `for` line is correct?

A
```
for (int i = 0; i <= static_cast<int>(a.size()); ++i)
```

B
```
for (int i = 0; i < static_cast<int>(a.size()); ++i)
```

C
```
for (int i = 1; i < static_cast<int>(a.size()); ++i)
```

D
```
for (int i = 0; i < a.size() - 1; ++i)
```

# Your Turn: Choose the Loop

The loop must visit **every index exactly once**, for any vector including an empty one. Which `for` line is correct?

A
```
for (int i = 0; i <= static_cast<int>(a.size()); ++i)
```

B
```
for (int i = 0; i < static_cast<int>(a.size()); ++i)
```

C
```
for (int i = 1; i < static_cast<int>(a.size()); ++i)
```

D
```
for (int i = 0; i < a.size() - 1; ++i)
```

**Raise your hand for A, B, C, or D.**

# Your Turn: Choose the Loop

The loop must visit **every index exactly once**, for any vector including an empty one. Which `for` line is correct?

A
```
for (int i = 0; i <= static_cast<int>(a.size()); ++i)
```

B
```
for (int i = 0; i < static_cast<int>(a.size()); ++i)
```

C
```
for (int i = 1; i < static_cast<int>(a.size()); ++i)
```

D
```
for (int i = 0; i < a.size() - 1; ++i)
```

**Answer: B.**

- **A:** <= also visits `a[a.size()]`, which is **one past the end**: out of bounds.
- **C:** starts at 1, so it **skips index 0**. Searching `{9, 4}` for 9 would fail.
- **D:** stops one element early and **skips the last one**. For an empty vector, `a.size() - 1` wraps around to a huge unsigned number.

# Your Turn: What If We Do Not Find the Target?

After the `for` loop we need a `return`. Where should `return -1;` go?

A. Inside the loop, in an `else` after the `if`: `if (a[i] == target) { return i; } else { return -1; }`

B. **After** the loop: `return -1;`

C. After the loop, but `return 0;`

D. Nowhere: if the loop ends, the function simply ends.

# Your Turn: What If We Do Not Find the Target?

After the `for` loop we need a `return`. Where should `return -1;` go?

A. Inside the loop, in an `else` after the `if`: `if (a[i] == target) { return i; } else { return -1; }`

B. **After** the loop: `return -1;`

C. After the loop, but `return 0;`

D. Nowhere: if the loop ends, the function simply ends.

**Raise your hand for A, B, C, or D.**

# Your Turn: What If We Do Not Find the Target?

After the `for` loop we need a `return`. Where should `return -1;` go?

A. Inside the loop, in an `else` after the `if`: `if (a[i] == target) { return i; } else { return -1; }`

B. **After** the loop: `return -1;`

C. After the loop, but `return 0;`

D. Nowhere: if the loop ends, the function simply ends.

**Answer: B.**

- **A:** the first mismatch returns $-1$ and the loop stops. `{7, 3, 9}`, target 9: after comparing with 7, it wrongly returns $-1$.
- **C:** 0 is a **valid index**: the caller cannot tell "found at index 0" from "not found".
- **D:** a non-`void` function that reaches its end without `return` is **undefined behavior**.
- **B:** the line after the loop is reached **only if no element matched**.

# Linear Search: Complete Code

Look at the elements one at a time, from the front.

```
// Returns the index of the first appearance of target in a;
// returns -1 if target is not in a.
int linearSearch(const std::vector<int>& a, int target) {
    for (int i = 0; i < static_cast<int>(a.size()); ++i) {
        if (a[i] == target) { return i; } // found: stop at once
    }
    return -1;                                  // not found
}
```

**Question.** With `a = {7, 3, 9, 4}`: how many comparisons does `linearSearch(a, 9)` make? And `linearSearch(a, 5)`?

# Linear Search: Complete Code

Look at the elements one at a time, from the front.

```
// Returns the index of the first appearance of target in a;
// returns -1 if target is not in a.
int linearSearch(const std::vector<int>& a, int target) {
    for (int i = 0; i < static_cast<int>(a.size()); ++i) {
        if (a[i] == target) { return i; } // found: stop at once
    }
    return -1;                                  // not found
}
```

**Question.** With `a = {7, 3, 9, 4}`: how many comparisons does `linearSearch(a, 9)` make? And `linearSearch(a, 5)`?

**9:** 3 comparisons (it returns index 2). **5:** all 4, and it returns $-1$. In the worst case (absent, or last) it makes $n$ comparisons: $O(n)$. On average, about $(n+1)/2$ comparisons (next slide shows why). **It works on any data**: sorted or not, vector or linked list.

# Average Cost of Linear Search

**Setting.** The target is **in** the vector (size $n$). Assume the $n$ elements are **distinct** and each is equally likely to be the target. (With duplicates, the first occurrence stops the search earlier.) We want the **average number of comparisons** over all $n$ possible targets.

**Question.** How many comparisons does `linearSearch` make if the target is the 1st element? the 2nd? the $k$-th? the last?

# Average Cost of Linear Search

**Setting.** The target is **in** the vector (size $n$). Assume the $n$ elements are **distinct** and each is equally likely to be the target. (With duplicates, the first occurrence stops the search earlier.) We want the **average number of comparisons** over all $n$ possible targets.

**Question.** How many comparisons does `linearSearch` make if the target is the 1st element? the 2nd? the $k$-th? the last?

Finding the $k$-th element takes $k$ **comparisons**:

| target is the... | 1st | 2nd | 3rd | $\cdots$ | $n$-th (last) |
|---|---|---|---|---|---|
| comparisons | 1 | 2 | 3 | $\cdots$ | $n$ |

# Average Cost of Linear Search

**Setting.** The target is **in** the vector (size $n$). Assume the $n$ elements are **distinct** and each is equally likely to be the target. (With duplicates, the first occurrence stops the search earlier.) We want the **average number of comparisons** over all $n$ possible targets.

**Question.** How many comparisons does `linearSearch` make if the target is the 1st element? the 2nd? the $k$-th? the last?

Finding the $k$-th element takes $k$ **comparisons**:

| target is the... | 1st | 2nd | 3rd | $\cdots$ | $n$-th (last) |
|---|---|---|---|---|---|
| comparisons | 1 | 2 | 3 | $\cdots$ | $n$ |

**Average** = total comparisons over all $n$ targets, divided by $n$:

$$\text{average} = \frac{1 + 2 + 3 + \cdots + n}{n}.$$

# Average Cost of Linear Search: The Sum

**Question.** What is $S = 1 + 2 + 3 + \cdots + n$? Try $n = 4$ and $n = 100$ first.

# Average Cost of Linear Search: The Sum

**Question.** What is $S = 1 + 2 + 3 + \cdots + n$? Try $n = 4$ and $n = 100$ first.

**Write the sum twice, the second time in reverse order, and add column by column:**

$$\begin{array}{rcccccccccc} S = & 1 & + & 2 & + & 3 & + \cdots + & (n-1) & + & n \\ S = & n & + & (n-1) & + & (n-2) & + \cdots + & 2 & + & 1 \\ \hline 2S = & (n+1) & + & (n+1) & + & (n+1) & + \cdots + & (n+1) & + & (n+1) \end{array}$$

# Average Cost of Linear Search: The Sum

**Question.** What is $S = 1 + 2 + 3 + \cdots + n$? Try $n = 4$ and $n = 100$ first.

**Write the sum twice, the second time in reverse order, and add column by column:**

$$\begin{array}{rl} S = & 1 + 2 + 3 + \cdots + (n-1) + n \\ S = & n + (n-1) + (n-2) + \cdots + 2 + 1 \\ \hline 2S = & (n+1) + (n+1) + (n+1) + \cdots + (n+1) + (n+1) \end{array}$$

There are exactly $n$ columns, each summing to $n+1$: $2S = n(n+1)$, so $S = \dfrac{n(n+1)}{2}$ (true for **every** positive integer $n$, even or odd).

Therefore average $= \dfrac{n(n+1)/2}{n} = \boxed{\dfrac{n+1}{2}}$ comparisons.

# Average Cost of Linear Search: The Sum

**Question.** What is $S = 1 + 2 + 3 + \cdots + n$? Try $n = 4$ and $n = 100$ first.

**Write the sum twice, the second time in reverse order, and add column by column:**

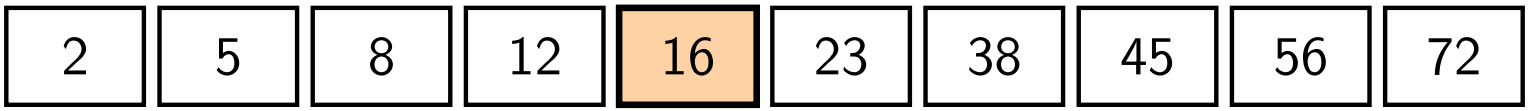
$$\begin{array}{rl} S = & 1 + 2 + 3 + \cdots + (n-1) + n \\ S = & n + (n-1) + (n-2) + \cdots + 2 + 1 \\ \hline 2S = & (n+1) + (n+1) + (n+1) + \cdots + (n+1) + (n+1) \end{array}$$

There are exactly $n$ columns, each summing to $n+1$: $2S = n(n+1)$, so $S = \dfrac{n(n+1)}{2}$ (true for **every** positive integer $n$, even or odd).

Therefore average $= \dfrac{n(n+1)/2}{n} = \boxed{\dfrac{n+1}{2}}$ comparisons. **Check.** $n = 4$: $\frac{1+2+3+4}{4} = 2.5 = \frac{4+1}{2}$. $n = 100$: 50.5. On average we scan about **half** of the vector: still $O(n)$. If the target may be **absent** (always $n$ comparisons), the average is higher but still $O(n)$.

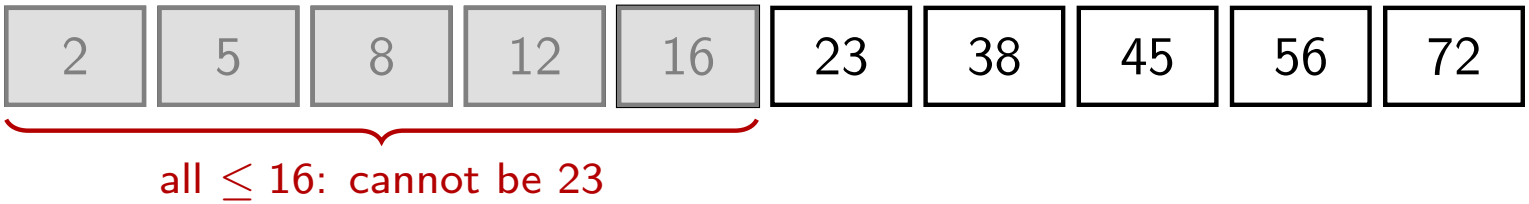
# Why Sorted Data Helps

**Question.** In the sorted array below we look at the middle element, 16, while searching for 23. What do we learn about the elements to the **left** of 16?



# Why Sorted Data Helps

**Question.** In the sorted array below we look at the middle element, 16, while searching for 23. What do we learn about the elements to the **left** of 16?




**All of them are** $\leq 16 < 23$. One comparison eliminates 16 and everything to its left: about **half** of the array. In an **unsorted** array, the middle element tells us **nothing** about the other elements: we cannot skip anything.

# Binary Search: Function Header and the Range

**Specification.** The vector `a` is **sorted**. Return the index of `target` if it is found; otherwise return $-1$.

**Header.** Same as linear search: an `int` index comes back; the vector is passed by `const` reference.

```
int binarySearch(const std::vector<int>& a, int target)
```

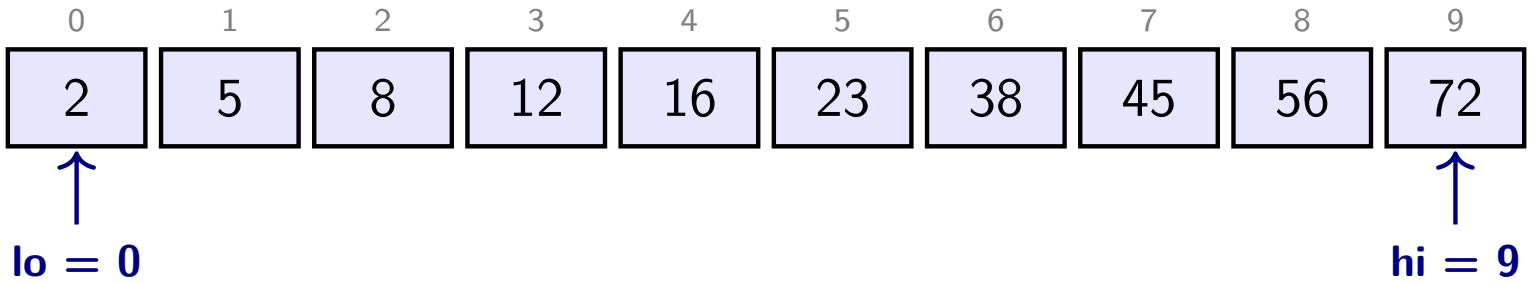
# Binary Search: Function Header and the Range

**Specification.** The vector `a` is **sorted**. Return the index of `target` if it is found; otherwise return $-1$.

**Header.** Same as linear search: an `int` index comes back; the vector is passed by `const` reference.

```
int binarySearch(const std::vector<int>& a, int target)
```

**The range.** Binary search keeps track of the part of the vector that **can still contain the target**: `lo` is its **lowest index**, `hi` is its **highest index**. At the start it is the whole vector: `lo = 0`, `hi = a.size() - 1`.



# Building It: `mid` and the Three Cases

**Step 1.** Set up the range: the whole vector.

```
int binarySearch(const std::vector<int>& a, int target) {
    int lo = 0, hi = static_cast<int>(a.size()) - 1; // range a[lo..hi]
}
```

# Building It: `mid` and the Three Cases

**Step 2. The key:** while the range is not empty, compute `mid` **from** `lo` **and** `hi`: the middle index of the range.

```
int binarySearch(const std::vector<int>& a, int target) {
    int lo = 0, hi = static_cast<int>(a.size()) - 1; // range a[lo..hi]
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        // ...
    }
}
```

# Building It: `mid` and the Three Cases

**Step 3.** Compare the element **at index `mid`** with `target`. There are three possibilities. **First:** `a[mid] == target`. We found it: return `mid`.

```
int binarySearch(const std::vector<int>& a, int target) {
    int lo = 0, hi = static_cast<int>(a.size()) - 1; // range a[lo..hi]
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == target) { return mid; } // found
        // ...
    }
}
```

# Building It: `mid` and the Three Cases

**Step 4. Second:** `a[mid] > target`. The target is smaller than `a[mid]`, so (the data is sorted) it can only be in the **left** part. `mid` is already checked, so the new `hi` is `mid - 1`.

```
int binarySearch(const std::vector<int>& a, int target) {
    int lo = 0, hi = static_cast<int>(a.size()) - 1; // range a[lo..hi]
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == target) { return mid; } // found
        if (a[mid] > target) { hi = mid - 1; } // go left
        // ...
    }
}
```

# Building It: `mid` and the Three Cases

**Step 5. Third:** the only case left is `a[mid] < target`. We do not need to write that condition: `else` already means it. The target can only be in the **right** part: the new `lo` is `mid + 1`.

```
int binarySearch(const std::vector<int>& a, int target) {
    int lo = 0, hi = static_cast<int>(a.size()) - 1; // range a[lo..hi]
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == target) { return mid; } // found
        if (a[mid] > target) { hi = mid - 1; } // go left
        else                 { lo = mid + 1; } // a[mid]<target: go right
    }
}
```

# Building It: `mid` and the Three Cases

**Step 6. When does the loop stop?** Each update shrinks the range. When the range is empty (`lo > hi`) the loop ends: the target is not in the vector.

```
int binarySearch(const std::vector<int>& a, int target) {
    int lo = 0, hi = static_cast<int>(a.size()) - 1; // range a[lo..hi]
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == target) { return mid; } // found
        if (a[mid] > target) { hi = mid - 1; } // go left
        else                 { lo = mid + 1; } // a[mid]<target: go right
    }
    // after the loop: ???
}
```

# Your Turn: How Do We Update `lo` or `hi`?

We just found `a[mid] > target`. The target (if present) is to the **left** of `mid`. Which update is correct?

A `lo = mid + 1;`

B `hi = mid - 1;`

C `hi = mid;`

D `lo = mid;`

# Your Turn: How Do We Update `lo` or `hi`?

We just found `a[mid] > target`. The target (if present) is to the **left** of `mid`. Which update is correct?

A `lo = mid + 1;`

B `hi = mid - 1;`

C `hi = mid;`

D `lo = mid;`

**Raise your hand for A, B, C, or D.**

# Your Turn: How Do We Update `lo` or `hi`?

We just found `a[mid] > target`. The target (if present) is to the **left** of `mid`. Which update is correct?

A `lo = mid + 1;`

B `hi = mid - 1;`

C `hi = mid;`

D `lo = mid;`

**Answer: B.**

- **A, D:** they move `lo`, which throws away the **left** part, where the target is.
- **C:** the correct side, but `mid` is kept although it was already checked. With `lo == hi == mid` nothing changes: an **infinite loop**.
- **B:** drops `mid` and everything to its right. The range **shrinks** every time. The mirror case (`a[mid] < target`) is `lo = mid + 1`.

# Your Turn: When Should the Loop Stop?

The range `lo..hi` contains the elements that are still possible. Which loop condition is correct?

A `while (lo < hi)`

B `while (lo <= hi)`

C `while (lo != hi)`

# Your Turn: When Should the Loop Stop?

The range `lo..hi` contains the elements that are still possible. Which loop condition is correct?

A `while (lo < hi)`

B `while (lo <= hi)`

C `while (lo != hi)`

**Raise your hand for A, B, or C.**

# Your Turn: When Should the Loop Stop?

The range `lo..hi` contains the elements that are still possible. Which loop condition is correct?

A `while (lo < hi)`

B `while (lo <= hi)`

C `while (lo != hi)`

**Answer: B.**

- The range is **empty** exactly when `lo > hi`, so we continue while `lo <= hi`. A range with **one element** has `lo == hi`: it still must be checked.
- **A:** stops before checking the last remaining element. `a = {5}`, target 5: `lo = hi = 0`, the loop never runs, and it wrongly returns $-1$.
- **C:** when the range becomes empty we have `lo > hi`, not `lo == hi`, so the loop would **keep going**. An empty vector gives `lo = 0, hi = -1`: it would read `a[0]` of an empty vector.

# After the Loop: Can We Omit `return -1`?

```
while (lo <= hi) {
    ... // found: return mid
}
// what runs here, and what should it do?
```

**Question.** When does execution leave the loop, and can we leave out `return -1;`?

# After the Loop: Can We Omit `return -1`?

```
while (lo <= hi) {
    ... // found: return mid
}
// what runs here, and what should it do?
```

**Question.** When does execution leave the loop, and can we leave out `return -1;`?

**When:** a successful search leaves through `return mid;`, so the loop **ends normally only when** `lo > hi`: the range is empty, the target is **not in the vector**.

# After the Loop: Can We Omit `return -1`?

```
while (lo <= hi) {
    ... // found: return mid
}
// what runs here, and what should it do?
```

**Question.** When does execution leave the loop, and can we leave out `return -1;`?

**When:** a successful search leaves through `return mid;`, so the loop **ends normally only when** `lo > hi`: the range is empty, the target is **not in the vector**.

**No, we cannot omit it.** Without it, control reaches the closing brace of an `int` function with **no return value**: **undefined behavior** (compilers warn: "control reaches end of non-void function"). The program is **no longer guaranteed to behave correctly**: the caller does not reliably get the signal "not found". The specification says: return $-1$.

# Binary Search: Complete Code

```cpp
// a must be sorted; returns an index of target, or -1
int binarySearch(const std::vector<int>& a, int target) {
    int lo = 0, hi = static_cast<int>(a.size()) - 1; // range a[lo..hi]
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == target) { return mid; } // found
        if (a[mid] > target) { hi = mid - 1; } // drop mid, right part
        else                 { lo = mid + 1; } // a[mid]<target: go right
    }
    return -1; // empty range
}
```

- `mid = lo + (hi - lo) / 2` instead of `(lo + hi) / 2`: the sum `lo + hi` can overflow `int`; the subtraction cannot (next slides).
- The $+1$ **and** $-1$ matter: `mid` has been checked, so it must leave the range.
- Assumption: the vector has at most `INT_MAX` elements (so `a.size()` fits in an `int`). An empty vector gives `hi = -1`, and the loop does not run.

# Why `lo + (hi - lo) / 2` and Not `(lo + hi) / 2`?

Both compute the same middle. But `int` has a largest value: `INT_MAX` $= 2\,147\,483\,647$.

**Question.** Let `hi = INT_MAX` and `lo = 1`. What happens in `(lo + hi) / 2`? What about `lo + (hi - lo) / 2`?

# Why `lo + (hi - lo) / 2` and Not `(lo + hi) / 2`?

Both compute the same middle. But `int` has a largest value: `INT_MAX` $= 2\,147\,483\,647$.

**Question.** Let `hi = INT_MAX` and `lo = 1`. What happens in `(lo + hi) / 2`? What about `lo + (hi - lo) / 2`?

**`(lo + hi) / 2`:** `lo + hi` $=$ `INT_MAX + 1` is **too big for an `int`**: overflow. Signed overflow is **undefined behavior**: the midpoint calculation is no longer reliable. (Wrapping to a negative number is one possible outcome.)

# Why `lo + (hi - lo) / 2` and Not `(lo + hi) / 2`?

Both compute the same middle. But `int` has a largest value: `INT_MAX` $= 2\,147\,483\,647$.

**Question.** Let `hi = INT_MAX` and `lo = 1`. What happens in `(lo + hi) / 2`? What about `lo + (hi - lo) / 2`?

**`(lo + hi) / 2`:** `lo + hi` $=$ `INT_MAX + 1` is **too big for an `int`**: overflow. Signed overflow is **undefined behavior**: the midpoint calculation is no longer reliable. (Wrapping to a negative number is one possible outcome.)

**`lo + (hi - lo) / 2`:** walk through the pieces (indices satisfy $0 \leq$ `lo` $\leq$ `hi`):

- `hi - lo` is between 0 and `hi`, and `(hi - lo) / 2` is smaller: **no overflow**.
- `lo + (hi - lo) / 2` $\leq$ `lo + (hi - lo)` $=$ `hi`: the result is **at most `hi`**, an `int` we already have, so it **cannot overflow**.

# Why `lo + (hi - lo) / 2` and Not `(lo + hi) / 2`?

Both compute the same middle. But `int` has a largest value: `INT_MAX` $= 2\,147\,483\,647$.

**Question.** Let `hi = INT_MAX` and `lo = 1`. What happens in `(lo + hi) / 2`? What about `lo + (hi - lo) / 2`?

**`(lo + hi) / 2`:** `lo + hi` $=$ `INT_MAX + 1` is **too big for an `int`**: overflow. Signed overflow is **undefined behavior**: the midpoint calculation is no longer reliable. (Wrapping to a negative number is one possible outcome.)

**`lo + (hi - lo) / 2`:** walk through the pieces (indices satisfy $0 \leq$ `lo` $\leq$ `hi`):

- `hi - lo` is between 0 and `hi`, and `(hi - lo) / 2` is smaller: **no overflow**.
- `lo + (hi - lo) / 2` $\leq$ `lo + (hi - lo)` $=$ `hi`: the result is **at most `hi`**, an `int` we already have, so it **cannot overflow**.

Mathematically $\texttt{lo} + \frac{\texttt{hi}-\texttt{lo}}{2} = \frac{\texttt{lo}+\texttt{hi}}{2}$: **same middle, no overflow**. (A binary search with this overflow bug sat in a widely used library for years.)

# Question: Can We Use `hi / 2 + lo / 2`?

It also avoids the overflow: `hi / 2 + lo / 2` is at most `hi`. So can we use it instead of `lo + (hi - lo) / 2`?

**Try it:** `lo = 1`, `hi = 3`. What does each formula give? Then try `lo = 3`, `hi = 3` (a range with one element).

# Question: Can We Use `hi / 2 + lo / 2`?

It also avoids the overflow: `hi / 2 + lo / 2` is at most `hi`. So can we use it instead of `lo + (hi - lo) / 2`?

**Try it:** `lo = 1`, `hi = 3`. What does each formula give? Then try `lo = 3`, `hi = 3` (a range with one element).

| | `lo + (hi - lo) / 2` | `hi / 2 + lo / 2` |
|---|---|---|
| `lo = 1, hi = 3` | $1 + \frac{2}{2} = \mathbf{2}$ | $1 + 0 = \mathbf{1}$ |
| `lo = 3, hi = 3` | $3 + 0 = \mathbf{3}$ | $1 + 1 = \mathbf{2}$ |

# Question: Can We Use `hi / 2 + lo / 2`?

It also avoids the overflow: `hi / 2 + lo / 2` is at most `hi`. So can we use it instead of `lo + (hi - lo) / 2`?

**Try it:** `lo = 1`, `hi = 3`. What does each formula give? Then try `lo = 3`, `hi = 3` (a range with one element).

| | `lo + (hi - lo) / 2` | `hi / 2 + lo / 2` |
|---|---|---|
| `lo = 1, hi = 3` | $1 + \frac{2}{2} = \mathbf{2}$ | $1 + 0 = \mathbf{1}$ |
| `lo = 3, hi = 3` | $3 + 0 = \mathbf{3}$ | $1 + 1 = \mathbf{2}$ |

**No.** Integer division **drops the remainder in each half separately**: when `lo` and `hi` are both odd, two remainders of $\frac{1}{2}$ are lost, which add up to a whole 1. The result is **one too small**.

- `lo = 1, hi = 3`: it picks 1, not the middle 2 (the search still shrinks, but is no longer balanced).
- `lo = hi = 3`: it gives 2, **outside the range** `[lo, hi]`. We would test an element that has already been excluded, and the search can fail.

Keep `lo + (hi - lo) / 2`: one division, applied to a number that already includes the remainder.

# What Do We Count? Search Steps

**Convention for the rest of this lecture.** One examined `mid` (one pass through the loop) is one **search step**.

**Question.** Searching for 23 in our 10-element array examines 16, 45, 23: 3 search steps. How many individual C++ tests against the target does the code perform?

# What Do We Count? Search Steps

**Convention for the rest of this lecture.** One examined `mid` (one pass through the loop) is one **search step**.

**Question.** Searching for 23 in our 10-element array examines 16, 45, 23: 3 search steps. How many individual C++ tests against the target does the code perform?

Each step does `a[mid] == target` and, if that is false, `a[mid] > target`: **at most 2 tests per step**. For 23 that is 5 tests ($2 + 2 + 1$).

# What Do We Count? Search Steps

**Convention for the rest of this lecture.** One examined `mid` (one pass through the loop) is one **search step**.

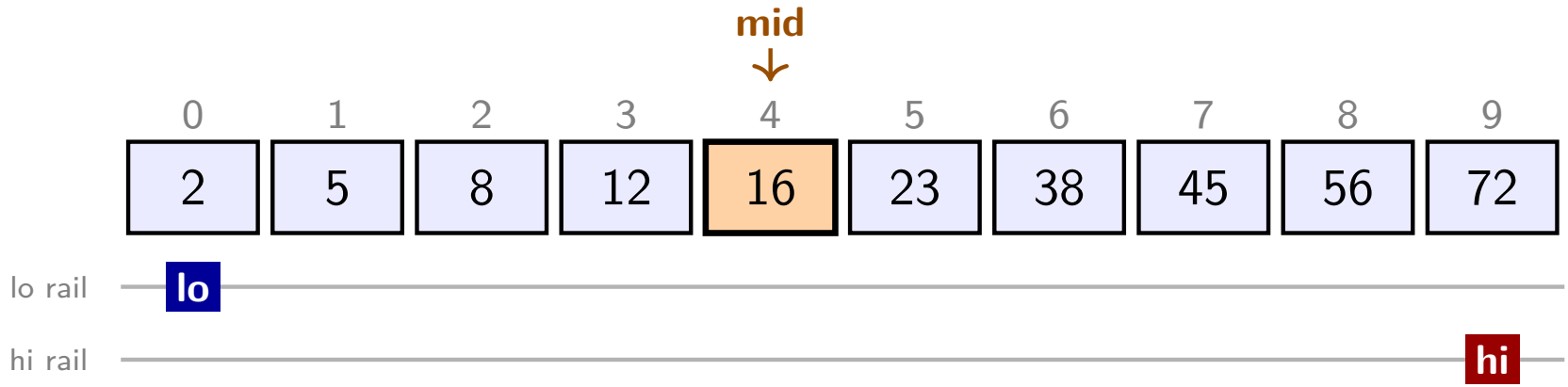
**Question.** Searching for 23 in our 10-element array examines 16, 45, 23: 3 search steps. How many individual C++ tests against the target does the code perform?

Each step does `a[mid] == target` and, if that is false, `a[mid] > target`: **at most 2 tests per step**. For 23 that is 5 tests ($2 + 2 + 1$).

A constant number of tests per step does not change the growth rate, so we count **search steps**. When a table or slide says "comparisons" for binary search, it means search steps (elements examined): 3 for 23, 4 for the absent 40, 4 for 72.

# Watch `lo` and `hi` Slide

Searching for **23**. Think of `lo` and `hi` as two **slide locks** on rails: after each search step one lock slides **past `mid`** and the part behind it is locked out.

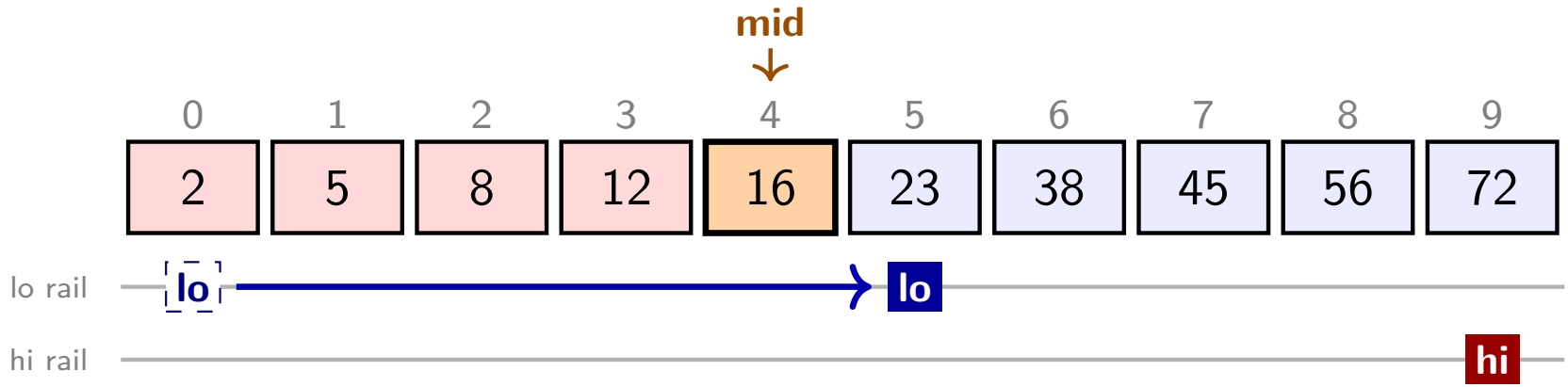



`lo = 0`, `hi = 9`: `mid = 0 + (9 - 0) / 2 = 4`, and `a[4] = 16`.

**Legend:** orange = `mid`; pink = about to be discarded; gray = already excluded.

# Watch `lo` and `hi` Slide

Searching for **23**. Think of `lo` and `hi` as two **slide locks** on rails: after each search step one lock slides **past `mid`** and the part behind it is locked out.

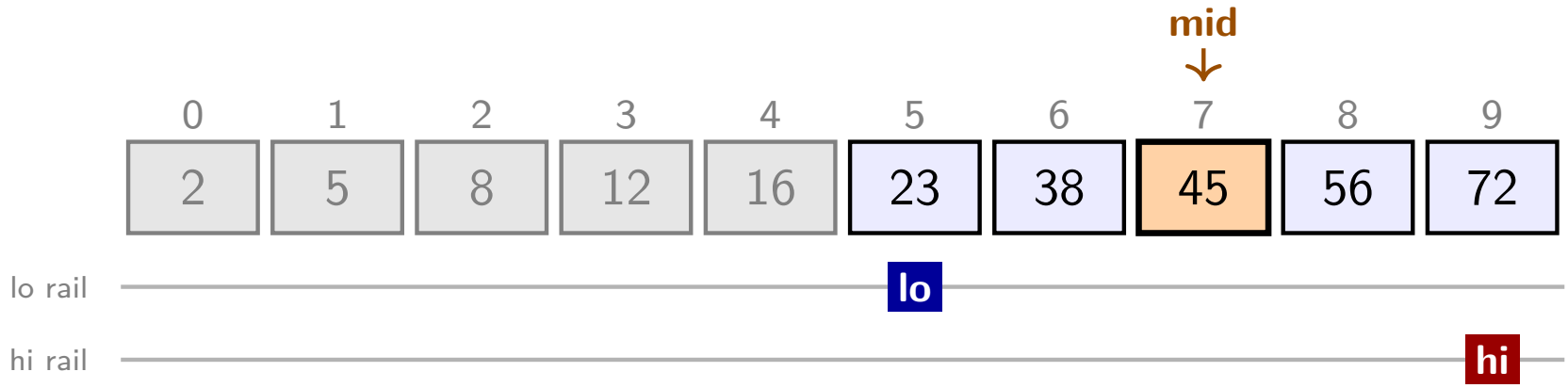



`a[mid] < target` ($16 < 23$): the target is to the **right**. Slide `lo` to `mid + 1 = 5` (`else { lo = mid + 1; }`). Everything left of the new `lo` is out.

**Legend:** orange = `mid`; pink = about to be discarded; gray = already excluded.

# Watch `lo` and `hi` Slide

Searching for **23**. Think of `lo` and `hi` as two **slide locks** on rails: after each search step one lock slides **past `mid`** and the part behind it is locked out.

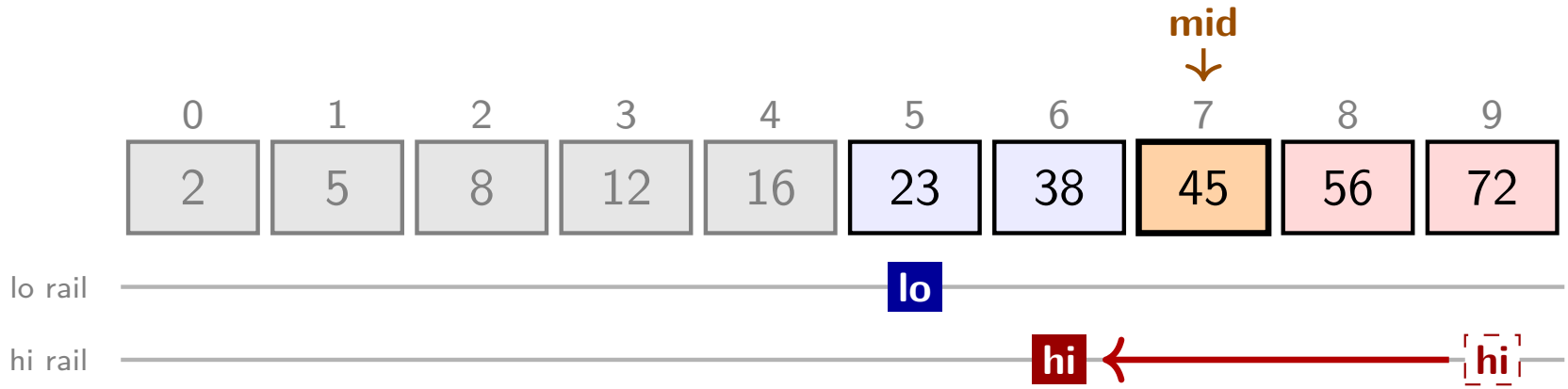



`lo = 5`, `hi = 9`: `mid = 5 + (9 - 5) / 2 = 7`, and `a[7] = 45`.

**Legend:** orange = `mid`; pink = about to be discarded; gray = already excluded.

# Watch `lo` and `hi` Slide

Searching for **23**. Think of `lo` and `hi` as two **slide locks** on rails: after each search step one lock slides **past `mid`** and the part behind it is locked out.

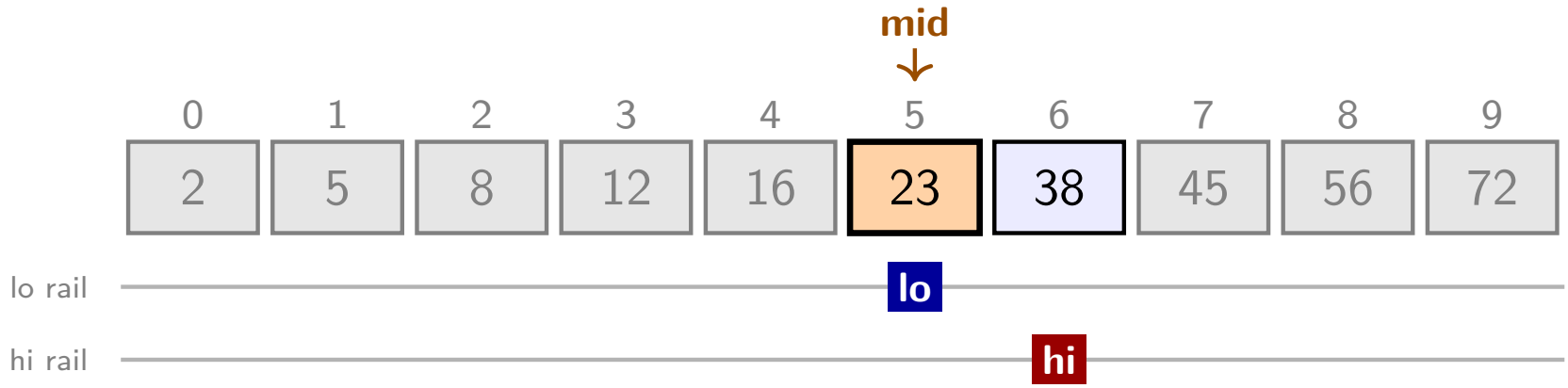



`a[mid] > target` ($45 > 23$): the target is to the **left**. Slide **`hi`** to `mid - 1 = 6` (`hi = mid - 1;`). Everything right of the new `hi` is out.

**Legend:** orange = `mid`; pink = about to be discarded; gray = already excluded.

# Watch `lo` and `hi` Slide

Searching for **23**. Think of `lo` and `hi` as two **slide locks** on rails: after each search step one lock slides **past `mid`** and the part behind it is locked out.

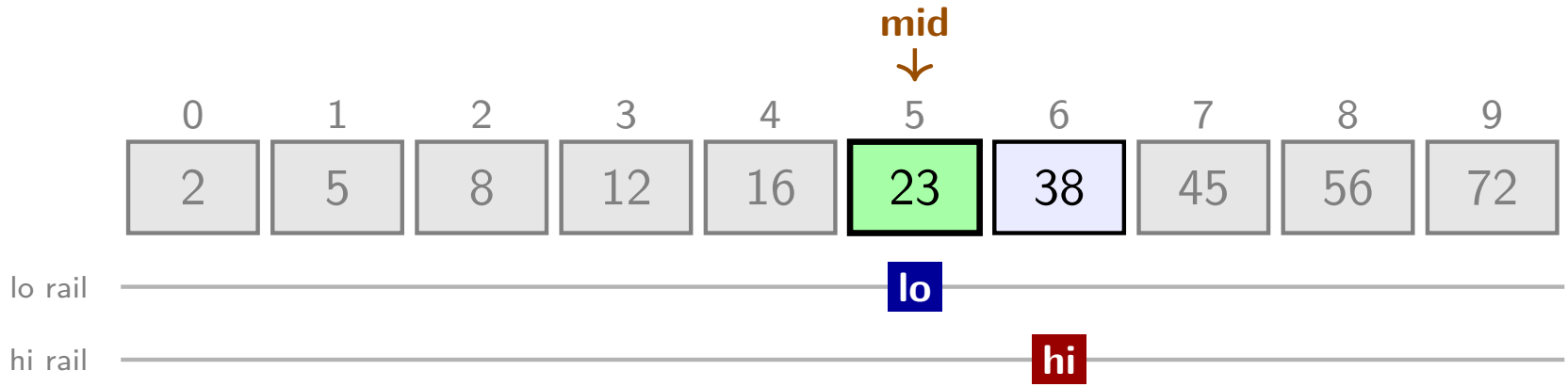



`lo = 5`, `hi = 6`: `mid = 5 + (6 - 5) / 2 = 5`, and `a[5] = 23`.

**Legend:** orange = `mid`; pink = about to be discarded; gray = already excluded.

# Watch `lo` and `hi` Slide

Searching for **23**. Think of `lo` and `hi` as two **slide locks** on rails: after each search step one lock slides **past `mid`** and the part behind it is locked out.




`a[mid] == target`: **found**, `return mid;` gives **5**. The candidate range shrank from 10 to 5 to 2 elements; the **third step** found the target.

**Legend:** orange = `mid`; pink = about to be discarded; gray = already excluded.

# Trace: Searching for 23

`a` has 10 elements; target **23**. Predict `mid` and the decision, then check.

| index | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| `a` | 2 | 5 | 8 | 12 | 16 | 23 | 38 | 45 | 56 | 72 |

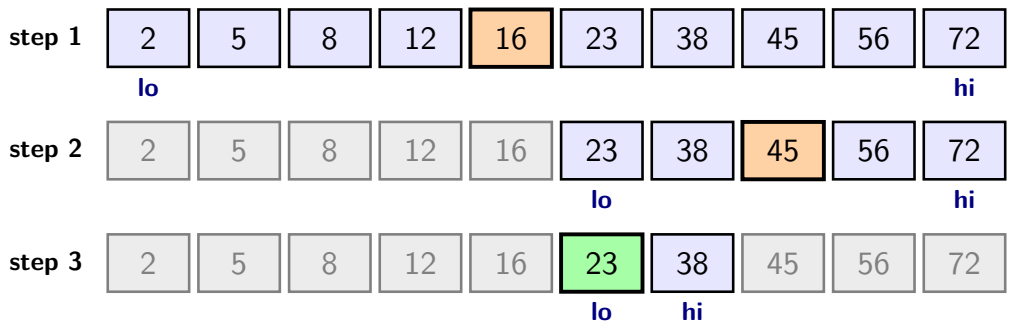
| step | `lo` | `hi` | `mid` | `a[mid]` | decision |
|---|---|---|---|---|---|
| 1 | 0 | 9 | 4 | 16 | |
| 2 | 5 | 9 | 7 | 45 | |
| 3 | 5 | 6 | 5 | 23 | |

# Trace: Searching for 23

`a` has 10 elements; target **23**. Predict `mid` and the decision, then check.

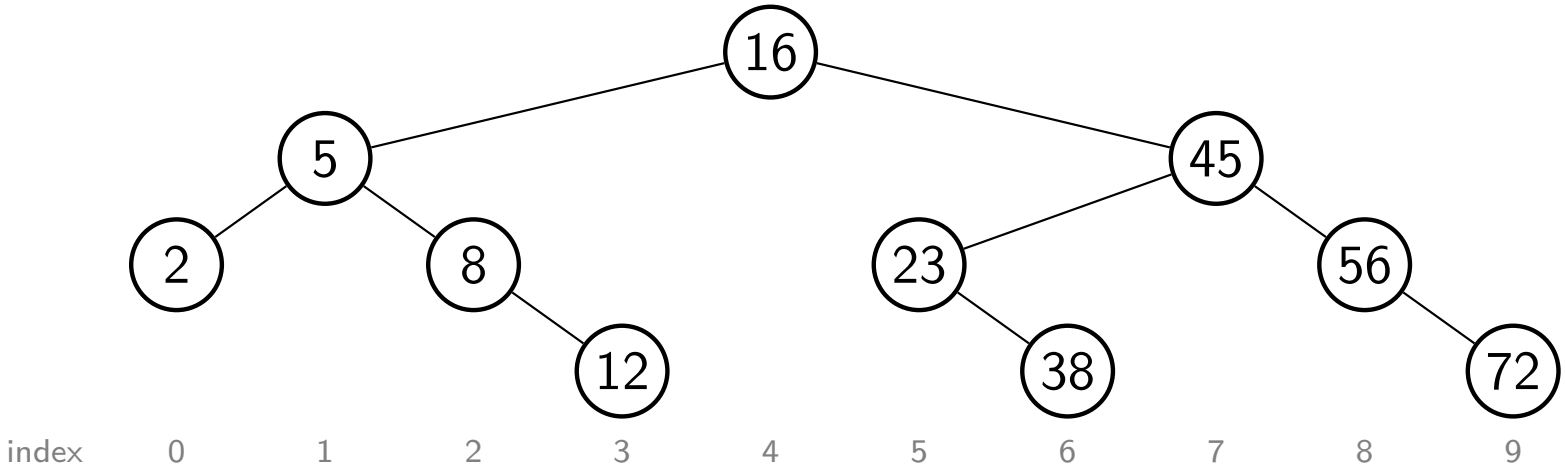
| index | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| `a` | 2 | 5 | 8 | 12 | 16 | 23 | 38 | 45 | 56 | 72 |

| step | `lo` | `hi` | `mid` | `a[mid]` | decision |
|---|---|---|---|---|---|
| 1 | 0 | 9 | 4 | 16 | $a[mid] <$ target: `lo = mid + 1` |
| 2 | 5 | 9 | 7 | 45 | $a[mid] >$ target: `hi = mid - 1` |
| 3 | 5 | 6 | 5 | 23 | **found**: return 5 |



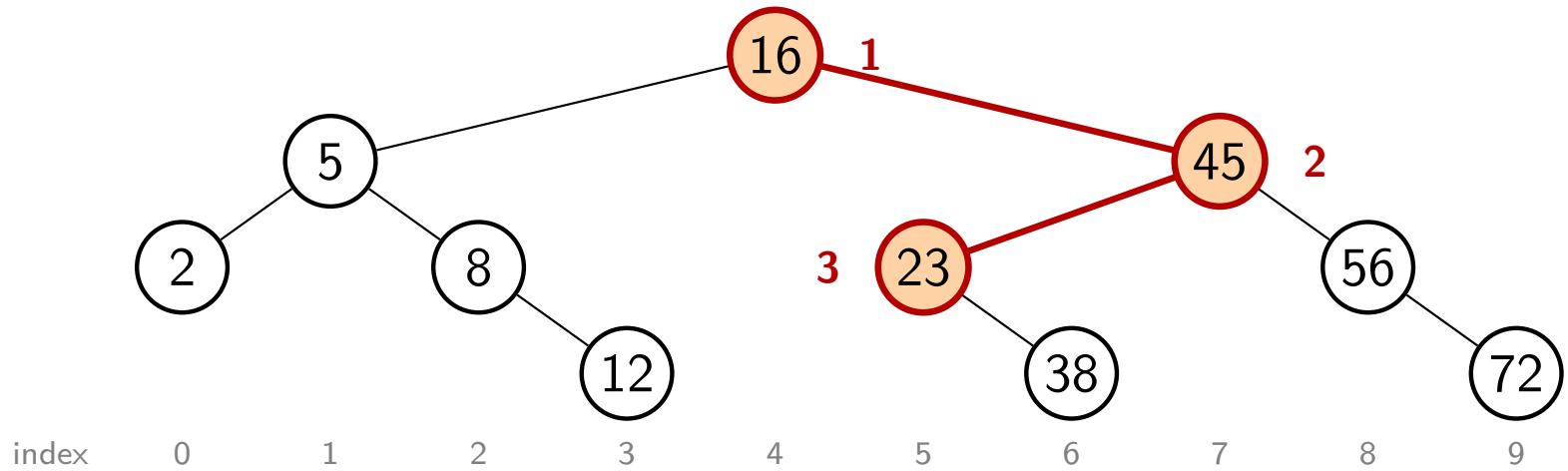
# Binary Search as a Tree

Every `mid` that binary search can look at is a **node**: the first `mid` is the **root**; the `mid` of the left (right) half is the **left (right) child**; and so on.



# Binary Search as a Tree

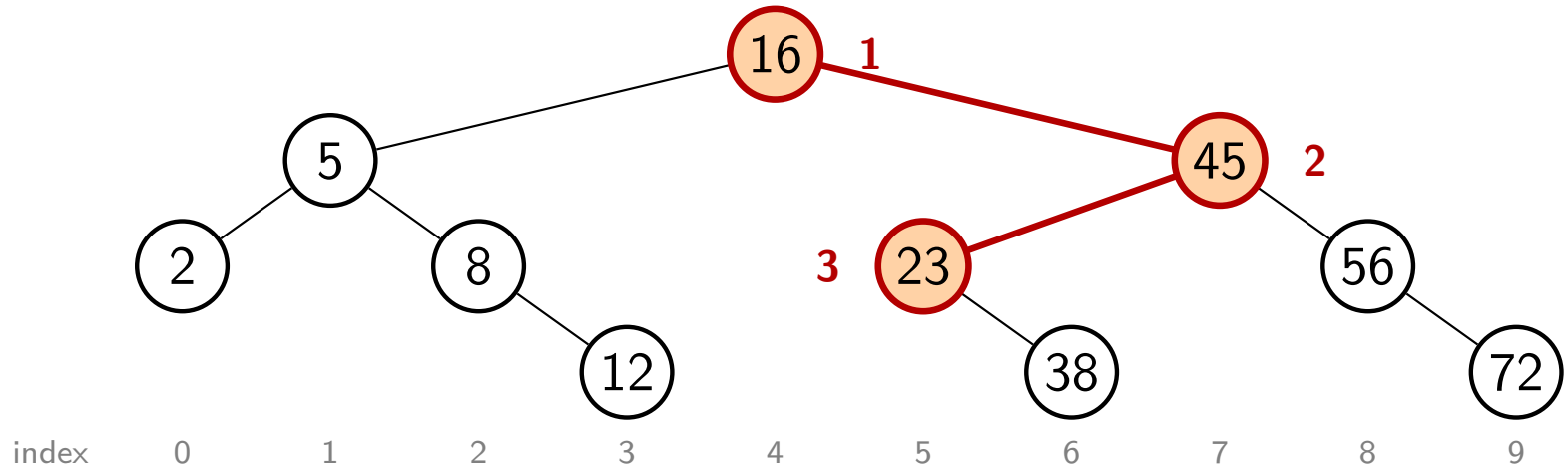
Every `mid` that binary search can look at is a **node**: the first `mid` is the **root**; the `mid` of the left (right) half is the **left (right) child**; and so on.




**Search for 23:** compare with 16 (1), go **right**; compare with 45 (2), go **left**; compare with 23 (3): found. Same 3 search steps as the trace: **each step moves down one level.**

# Binary Search as a Tree

Every `mid` that binary search can look at is a **node**: the first `mid` is the **root**; the `mid` of the left (right) half is the **left (right) child**; and so on.




**Search for 23:** compare with 16 (1), go **right**; compare with 45 (2), go **left**; compare with 23 (3): found. Same 3 search steps as the trace: **each step moves down one level.**

A **binary search tree**: for every node, its **left** subtree has values $\leq$ the node, its **right** subtree has values $>$ the node. This tree is **balanced** (we always chose the middle): about $\log_2 n$ levels, so $O(\log n)$. Another insertion order can give a chain of $n$ levels. We build such trees on Nov 24.

# Question: How Many Comparisons?

**(a)** Binary search on $n = 1\,000\,000$ sorted numbers: about how many search steps (elements examined), in the worst case?
**(b)** What if the array doubles to $2\,000\,000$?

| $n$ | **Linear (worst)** | **Binary (worst, steps)** |
|---:|---:|---|
| 10 | 10 | |
| 100 | 100 | |
| 1 000 | 1 000 | |
| 1 000 000 | 1 000 000 | |
| 1 000 000 000 | 1 000 000 000 | |

# Question: How Many Comparisons?

**(a)** Binary search on $n = 1\,000\,000$ sorted numbers: about how many search steps (elements examined), in the worst case?
**(b)** What if the array doubles to $2\,000\,000$?

| $n$ | **Linear (worst)** | **Binary (worst, steps)** |
|---:|---:|---:|
| 10 | 10 | 4 |
| 100 | 100 | 7 |
| 1 000 | 1 000 | 10 |
| 1 000 000 | 1 000 000 | 20 |
| 1 000 000 000 | 1 000 000 000 | 30 |

**(a)** 20. **(b)** 21: doubling the array adds **one** worst-case search step. Worst case is $\lfloor \log_2 n \rfloor + 1$ (I checked this by running every possible search for these sizes).

# Where Does the 10 Come From? ($n = 1000$)

**Question.** Powers of 2: what are $2^9$ and $2^{10}$? Where does 1000 fall between them?

# Where Does the 10 Come From? ($n = 1000$)

**Question.** Powers of 2: what are $2^9$ and $2^{10}$? Where does 1000 fall between them?

| $2^9$ | 1000 | $2^{10}$ |
|---|---|---|
| 512 | $\leq 1000 <$ | 1024 |

# Where Does the 10 Come From? ($n = 1000$)

**Question.** Powers of 2: what are $2^9$ and $2^{10}$? Where does 1000 fall between them?

| $2^9$ | 1000 | $2^{10}$ |
|---|---|---|
| 512 | $\leq 1000 <$ | 1024 |

So $9 < \log_2 1000 < 10$: indeed $\log_2 1000 \approx 9.97$, and $\lfloor \log_2 1000 \rfloor = \mathbf{9}$.

Halving 1000:
$1000 \to 500 \to 250 \to 125 \to 62 \to 31 \to 15 \to 7 \to 3 \to 1$. That is **9 halvings**, so $9 + 1 = \mathbf{10}$ elements are examined in the worst case (the last one is the single element left).

# Where Does the 10 Come From? ($n = 1000$)

**Question.** Powers of 2: what are $2^9$ and $2^{10}$? Where does 1000 fall between them?

| $2^9$ | 1000 | $2^{10}$ |
|---|---|---|
| 512 | $\leq 1000 <$ | 1024 |

So $9 < \log_2 1000 < 10$: indeed $\log_2 1000 \approx 9.97$, and $\lfloor \log_2 1000 \rfloor = \mathbf{9}$.

Halving 1000:
$1000 \to 500 \to 250 \to 125 \to 62 \to 31 \to 15 \to 7 \to 3 \to 1$. That is **9 halvings**, so $9 + 1 = \mathbf{10}$ elements are examined in the worst case (the last one is the single element left). **Rule:** worst case $= \lfloor \log_2 n \rfloor + 1$, where $2^{\lfloor \log_2 n \rfloor} \leq n < 2^{\lfloor \log_2 n \rfloor + 1}$. Doubling $n$ adds exactly one.

# Where the Numbers in the Table Come From

Find the two powers of 2 around $n$: $2^k \le n < 2^{k+1}$, then the worst case is $k+1$.

| $n$ | $2^k \le n < 2^{k+1}$ | $\log_2 n$ | **worst** $k+1$ |
|---:|---|---:|---:|
| 10 | $8 \le n < 16$ $(2^3, 2^4)$ | 3.32 | 4 |
| 100 | $64 \le n < 128$ $(2^6, 2^7)$ | 6.64 | 7 |
| 1 000 | $512 \le n < 1\,024$ $(2^9, 2^{10})$ | 9.97 | 10 |
| 1 000 000 | $524\,288 \le n < 1\,048\,576$ $(2^{19}, 2^{20})$ | 19.93 | 20 |
| 1 000 000 000 | $536\,870\,912 \le n < 1\,073\,741\,824$ $(2^{29}, 2^{30})$ | 29.90 | 30 |

A handy fact: $2^{10} = 1024 \approx 10^3$, so $2^{20} \approx 10^6$ and $2^{30} \approx 10^9$. Every factor of 1000 in $n$ adds only about 10 search steps.

# Why Is It $O(\log n)$?

Each step discards at least half of the remaining range. The range sizes are

$$n, \quad \frac{n}{2}, \quad \frac{n}{4}, \quad \frac{n}{8}, \quad \ldots, \quad 1$$

**Question:** how many times can you halve $n$ before you reach 1?

# Why Is It $O(\log n)$?

Each step discards at least half of the remaining range. The range sizes are

$$n, \quad \frac{n}{2}, \quad \frac{n}{4}, \quad \frac{n}{8}, \quad \ldots, \quad 1$$

**Question:** how many times can you halve $n$ before you reach 1?

About $\log_2 n$ times, because $n/2^k \leq 1$ exactly when $k \geq \log_2 n$. So there are about $\log_2 n$ loop iterations (at most $\lfloor \log_2 n \rfloor + 1$, as the table showed), and each iteration does a constant amount of work. So the total is $O(\log n)$.

**Recurrence form:** $T(n) = T(n/2) + O(1)$. (Next week's merge sort has $T(n) = 2T(n/2) + O(n)$.)

**Rule of thumb:** a loop that halves (or doubles) something each time, with $O(1)$ work per pass, is $O(\log n)$.

# Counting the Work: A Recurrence

Let $f(n)$ be the **number of operations** binary search needs for a sorted array of $n$ elements.

**Question.** In one loop iteration, what work do we do? Does the amount depend on $n$?

# Counting the Work: A Recurrence

Let $f(n)$ be the **number of operations** binary search needs for a sorted array of $n$ elements.

**Question.** In one loop iteration, what work do we do? Does the amount depend on $n$?

Compute `mid` (+, -, /), compare `a[mid]` with `target`, update `lo` or `hi`: a **constant** number of operations, call it $c$, **whatever $n$ is**. After that, the range is cut to (at most) half. So

$$f(n) = c + f(n/2), \qquad f(1) = c_2,$$

where $c_2$ is the constant work for a one-element array.

# Unrolling the Recurrence

**Suppose** $n = 2^m$ (so $m = \log_2 n$). Replace $f(n/2)$ by its own recurrence, again and again:

$$\begin{aligned} f(n) &= c + f(n/2) \\ &= c + c + f(n/4) = 2c + f(n/4) \\ &= 3c + f(n/8) \\ &\vdots \\ &= m\,c + f(n/2^m) = m\,c + f(1). \end{aligned}$$

# Unrolling the Recurrence

**Suppose** $n = 2^m$ (so $m = \log_2 n$). Replace $f(n/2)$ by its own recurrence, again and again:

$$\begin{aligned} f(n) &= c + f(n/2) \\ &= c + c + f(n/4) = 2c + f(n/4) \\ &= 3c + f(n/8) \\ &\vdots \\ &= m\,c + f(n/2^m) = m\,c + f(1). \end{aligned}$$

So $f(n) = c \log_2 n + c_2$: $O(\log n)$. Each halving costs the constant $c$, and we can halve $n = 2^m$ exactly $m$ times before one element is left.

# When $n$ Is Not a Power of 2

**Question.** Our array has $n = 7$ elements. 7 is not $2^m$. How many halvings can binary search need at most? What about $n = 100$?

# When $n$ Is Not a Power of 2

**Question.** Our array has $n = 7$ elements. 7 is not $2^m$. How many halvings can binary search need at most? What about $n = 100$?

Find the power of 2 just above $n$: $2^{k-1} < n \leq 2^k$, so $k = \lceil \log_2 n \rceil$.

| $n$ | $2^{k-1} < n \leq 2^k$ | $k = \lceil \log_2 n \rceil$ | range sizes (at most) |
|---|---|---|---|
| 7 | $4 < 7 \leq 8$ | 3 | $7 \to 3 \to 1$ |
| 100 | $64 < 100 \leq 128$ | 7 | $100 \to 50 \to 25 \to 12 \to 6 \to 3 \to 1$ |

# When $n$ Is Not a Power of 2

**Question.** Our array has $n = 7$ elements. 7 is not $2^m$. How many halvings can binary search need at most? What about $n = 100$?

Find the power of 2 just above $n$: $2^{k-1} < n \leq 2^k$, so $k = \lceil \log_2 n \rceil$.

| $n$ | $2^{k-1} < n \leq 2^k$ | $k = \lceil \log_2 n \rceil$ | range sizes (at most) |
|---|---|---|---|
| 7 | $4 < 7 \leq 8$ | 3 | $7 \to 3 \to 1$ |
| 100 | $64 < 100 \leq 128$ | 7 | $100 \to 50 \to 25 \to 12 \to 6 \to 3 \to 1$ |

**Why this is enough.** Searching $n$ elements is **no more work** than searching $2^k \geq n$ elements (a smaller range can only finish sooner or at the same time). We already know the power-of-2 case:

$$f(n) \leq f(2^k) = c\,k + c_2 = c\lceil \log_2 n \rceil + c_2 < c \log_2 n + c + c_2.$$

# When $n$ Is Not a Power of 2

**Question.** Our array has $n = 7$ elements. 7 is not $2^m$. How many halvings can binary search need at most? What about $n = 100$?

Find the power of 2 just above $n$: $2^{k-1} < n \leq 2^k$, so $k = \lceil \log_2 n \rceil$.

| $n$ | $2^{k-1} < n \leq 2^k$ | $k = \lceil \log_2 n \rceil$ | range sizes (at most) |
|---|---|---|---|
| 7 | $4 < 7 \leq 8$ | 3 | $7 \to 3 \to 1$ |
| 100 | $64 < 100 \leq 128$ | 7 | $100 \to 50 \to 25 \to 12 \to 6 \to 3 \to 1$ |

**Why this is enough.** Searching $n$ elements is **no more work** than searching $2^k \geq n$ elements (a smaller range can only finish sooner or at the same time). We already know the power-of-2 case:

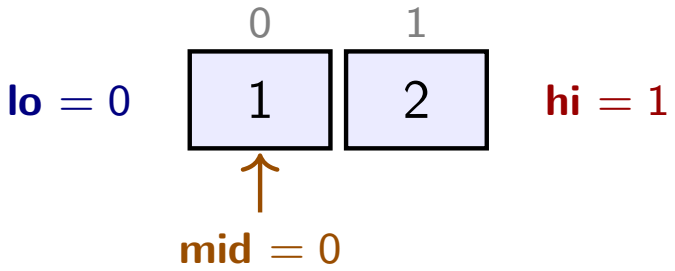
$$f(n) \leq f(2^k) = c\,k + c_2 = c\lceil \log_2 n \rceil + c_2 < c \log_2 n + c + c_2.$$

So $f(n)$ is **at most about** $\log_2 n$ **steps** times a constant, plus a constant: $f(n) = O(\log n)$. This matches the guessing game: 100 numbers, $k = 7$, at most 7 guesses.

# Question: A Bug Hiding in the Update

A student writes `lo = mid; (no + 1)` in the first branch. Run it on `a = {1, 2}`, target **2**. `lo = 0, hi = 1`.

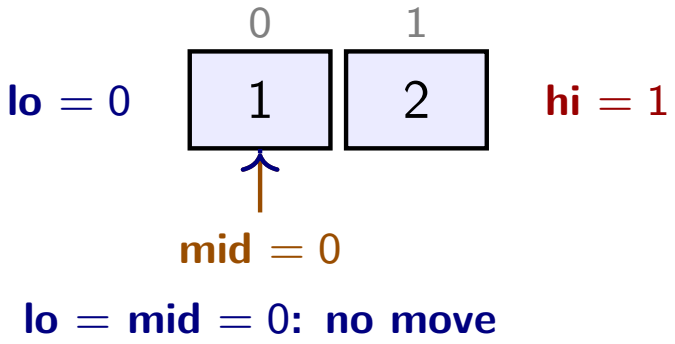
**(a)** What is `mid`? What does the code do next?
**(b)** What happens on the next iteration?



# Question: A Bug Hiding in the Update

A student writes `lo = mid;` (no `+ 1`) in the first branch. Run it on `a = {1, 2}`, target **2**. `lo = 0, hi = 1`.

**(a)** What is `mid`? What does the code do next?
**(b)** What happens on the next iteration?




**(a)** `mid = 0 + (1 - 0)/2 = 0`. `a[0] = 1 < 2`, so `lo = mid = 0`: **nothing changed**.
**(b)** The same state repeats forever: an **infinite loop**. The range never shrinks because `mid` was not removed. (I ran it: after 6 iterations, `lo = 0, hi = 1, mid = 0` every time.)

**Rule:** every step must **shrink the range**. `mid` has just been tested, so drop it: `mid + 1` or `mid - 1`.

# Does the Data Structure Matter?

Binary search needs to jump to the middle **in one step**.

**Question:** you have a sorted **linked list** of $n$ numbers. Can you do binary search on it in $O(\log n)$ time?

# Does the Data Structure Matter?

Binary search needs to jump to the middle **in one step**.

**Question:** you have a sorted **linked list** of $n$ numbers. Can you do binary search on it in $O(\log n)$ time?

**No.** A linked list has no `a[mid]`: reaching the middle node means following $n/2$ `next` pointers, which is $O(n)$ work. In a straightforward implementation the successive midpoints cost about $n/2 + n/4 + \cdots = O(n)$ hops in total: only $O(\log n)$ comparisons, but the pointer-following ruins the $O(\log n)$ time.

**Needs random access:** an array or a `std::vector`, where `a[i]` is $O(1)$ (like your `MiniVector`).

**Two requirements:** (1) the data is **sorted**; (2) we can reach any element in $O(1)$.

# Common Mistakes

- **Binary search on unsorted data**: the answer is wrong, and nothing warns you.
- **Updating with `lo = mid` or `hi = mid` in the `lo <= hi` version**: the loop can run forever.
- **Using `lo < hi` where `lo <= hi` is needed**: misses the case of one element left.
- **`(lo + hi) / 2` for huge arrays**: the sum can overflow. Use `lo + (hi - lo) / 2`.
- **Expecting the first duplicate**: plain binary search returns *some* match, not necessarily the first. (Optional supplement: `lowerBound`.)
- **Calling a loop that halves "$O(n)$"**: true as an upper bound but loose; the tighter bound is $O(\log n)$. Do not infer complexity from nesting alone: count each loop's iterations and the total work. Two nested loops that each run $n$ times do $\Theta(n^2)$ work.

# Summary and Preview

- **Linear search:** any data, $O(n)$. **Binary search:** sorted data with random access, $O(\log n)$.
- Binary search keeps the invariant "the target, if present, is in `lo..hi`" and shrinks the range every step.
- **Doubling the data adds one search step.**

## Coming up

**Nov 17:** Sorting I: bubble, selection, and insertion sort, and why they are $O(n^2)$. Exercise 9 (Sorting Trace) posted.
**Nov 19:** merge sort, using the same halving idea.

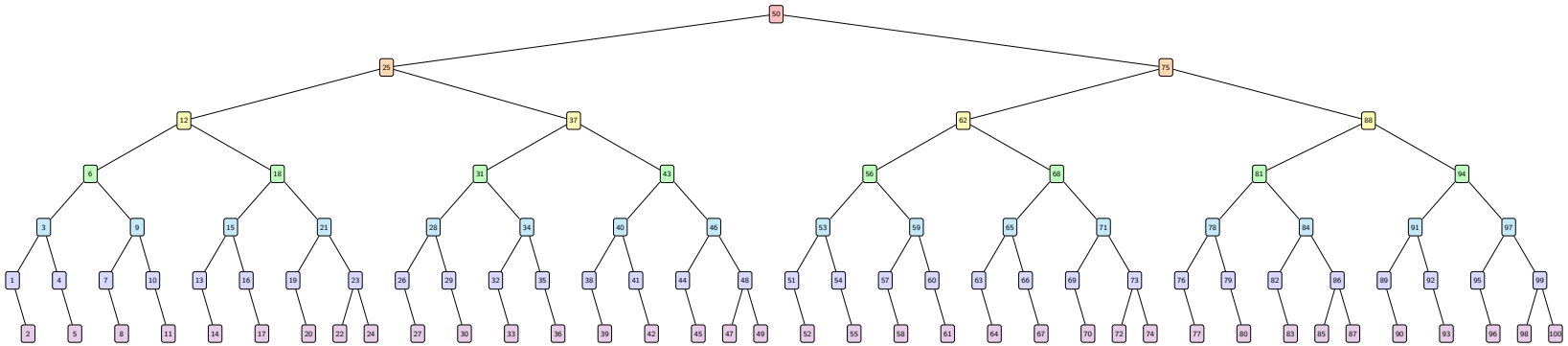
# Backup Slides

Extra material for questions and for review, not part of the main lecture:

- A second trace: searching for 40 (not present).
- Sort first? When sorting pays off.
- Growth rates we have met.
- Practice questions with answers.

# Backup: The Whole Tree for 1 to 100

All 100 numbers; the middle of each range is the node (the same rule as `mid = lo + (hi - lo) / 2`). Level sizes: $1, 2, 4, 8, 16, 32, 37$.



# Trace: Searching for 40 (Not Present)

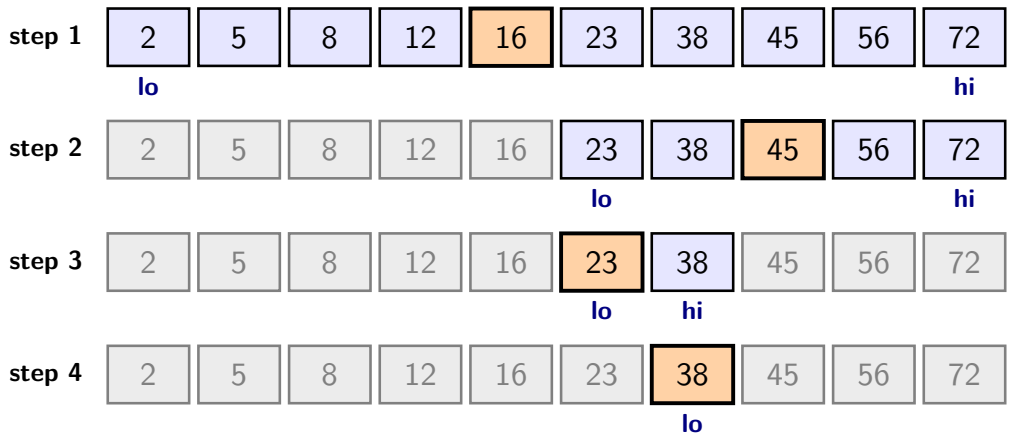
Same array, target **40**. What happens when the target is absent?

| step | `lo` | `hi` | `mid` | `a[mid]` | decision |
|---|---|---|---|---|---|
| 1 | 0 | 9 | 4 | 16 | |
| 2 | 5 | 9 | 7 | 45 | |
| 3 | 5 | 6 | 5 | 23 | |
| 4 | 6 | 6 | 6 | 38 | |

# Trace: Searching for 40 (Not Present)

Same array, target **40**. What happens when the target is absent?

| step | lo | hi | mid | a[mid] | decision |
|---|---|---|---|---|---|
| 1 | 0 | 9 | 4 | 16 | $a[mid] <$ target: `lo = mid + 1` |
| 2 | 5 | 9 | 7 | 45 | $a[mid] >$ target: `hi = mid - 1` |
| 3 | 5 | 6 | 5 | 23 | $a[mid] <$ target: `lo = mid + 1` |
| 4 | 6 | 6 | 6 | 38 | $a[mid] <$ target: `lo = mid + 1` |




After step 4, `lo = 7` and `hi = 6`: the range is **empty**, the loop ends, and the function returns $-1$. Four search steps (elements examined) for 10 elements.

# Question: Sort First?

A `vector` holds $n = 1\,000\,000$ numbers in **random order**. Sorting costs about $n \log_2 n \approx 20\,000\,000$ steps (Class 20 will show this).

**(a)** You will search for **one** number. Sort first, or just do a linear search?
**(b)** You will search for **1 000 different** numbers. Sort first?

# Question: Sort First?

A `vector` holds $n = 1\,000\,000$ numbers in **random order**. Sorting costs about $n \log_2 n \approx 20\,000\,000$ steps (Class 20 will show this).

**(a)** You will search for **one** number. Sort first, or just do a linear search?
**(b)** You will search for **1 000 different** numbers. Sort first?

**(a)** Linear search: $\approx 10^6$ steps, which is less than the $2 \times 10^7$ steps to sort.

**(b)** 1 000 linear searches: $1000 \times 10^6 = 10^9$ steps. Sort once ($2 \times 10^7$), then 1 000 binary searches ($1000 \times 20 = 2 \times 10^4$): about $2 \times 10^7$ in total. **Sorting wins by a factor of 50.**

**Lesson:** sorting is an investment. It pays off when you search many times. Complexity helps you decide. (These are approximate step counts, not measured running times.)

# Growth Rates We Have Met

Illustrative step counts for $n = 1\,000\,000$ (constant factors ignored):

| **Growth** | **Example** | **Steps** |
|---|---|---|
| $O(1)$ | `v[i]`, push to a stack | 1 |
| $O(\log n)$ | binary search | 20 |
| $O(n)$ | linear search, traversing a list | $1\,000\,000$ |
| $O(n \log n)$ | sorting well (merge sort, Class 22) | $\approx 20\,000\,000$ |
| $O(n^2)$ | nested loops, simple sorts (Class 21) | $10^{12}$ |

# Growth Rates We Have Met

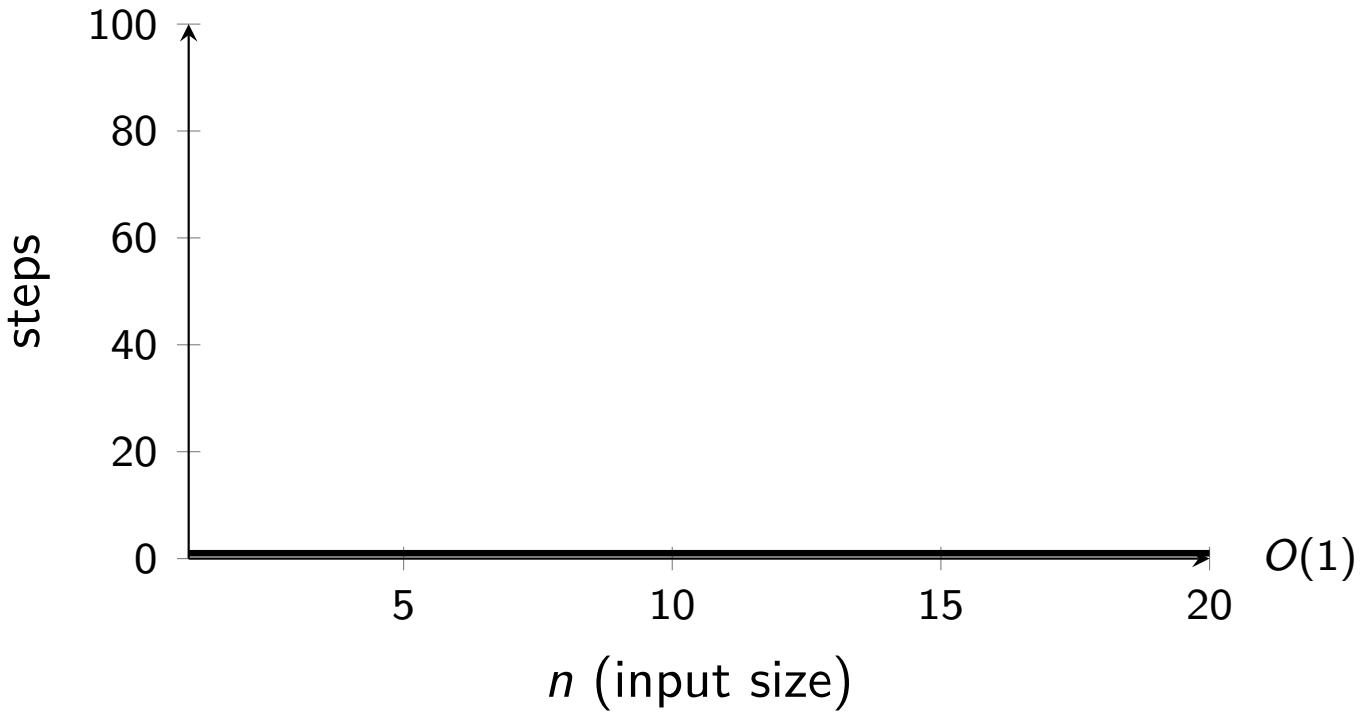
Illustrative step counts for $n = 1\,000\,000$ (constant factors ignored):

| Growth | Example | Steps |
|---|---|---|
| $O(1)$ | `v[i]`, push to a stack | 1 |
| $O(\log n)$ | binary search | 20 |
| $O(n)$ | linear search, traversing a list | $1\,000\,000$ |
| $O(n \log n)$ | sorting well (merge sort, Class 22) | $\approx 20\,000\,000$ |
| $O(n^2)$ | nested loops, simple sorts (Class 21) | $10^{12}$ |

**Reading the table:** $\log n$ grows very slowly, $n \log n$ is fine, $n^2$ is already too slow for a million items. At $10^9$ simple operations per second, $10^{12}$ steps takes about 17 minutes; $2 \times 10^7$ steps takes 0.02 seconds.

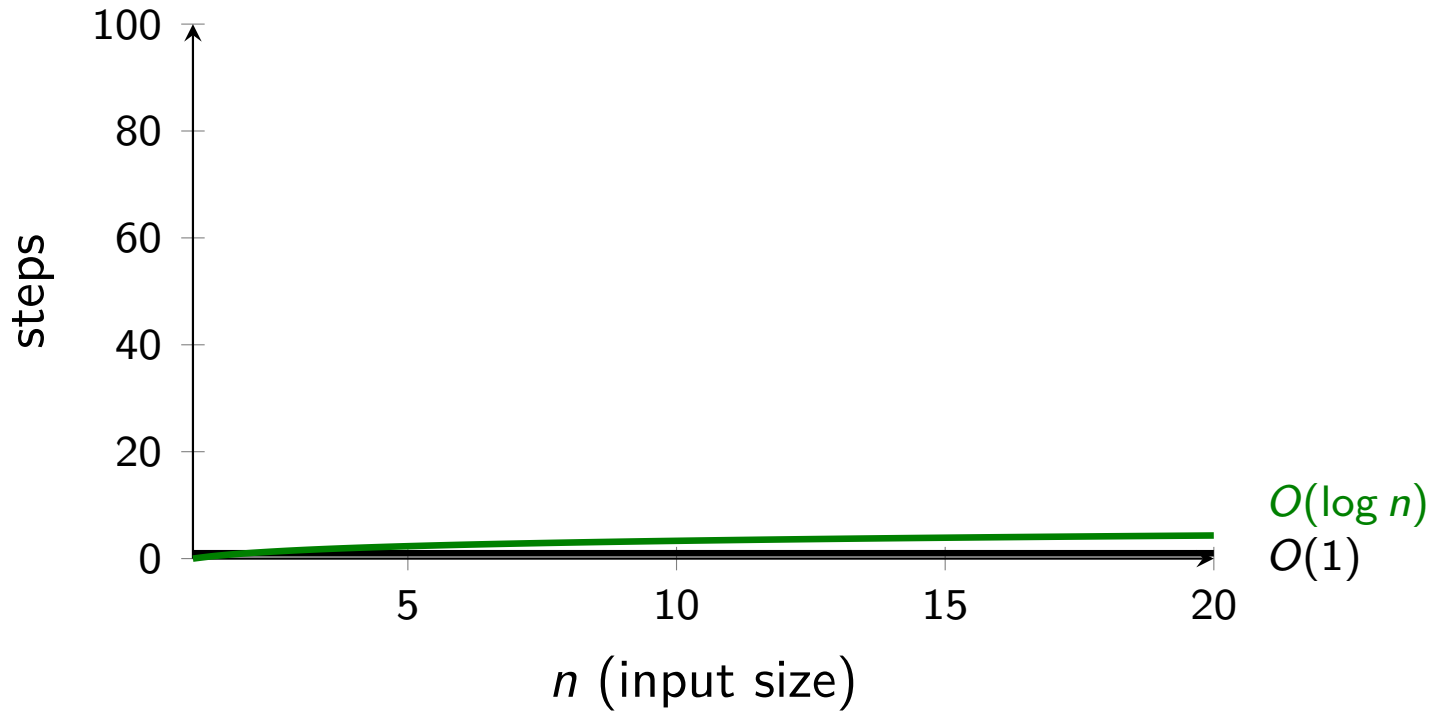
# Growth Rate Curves

The same five growth rates drawn as functions of $n$ (number of steps on the vertical axis):



# Growth Rate Curves

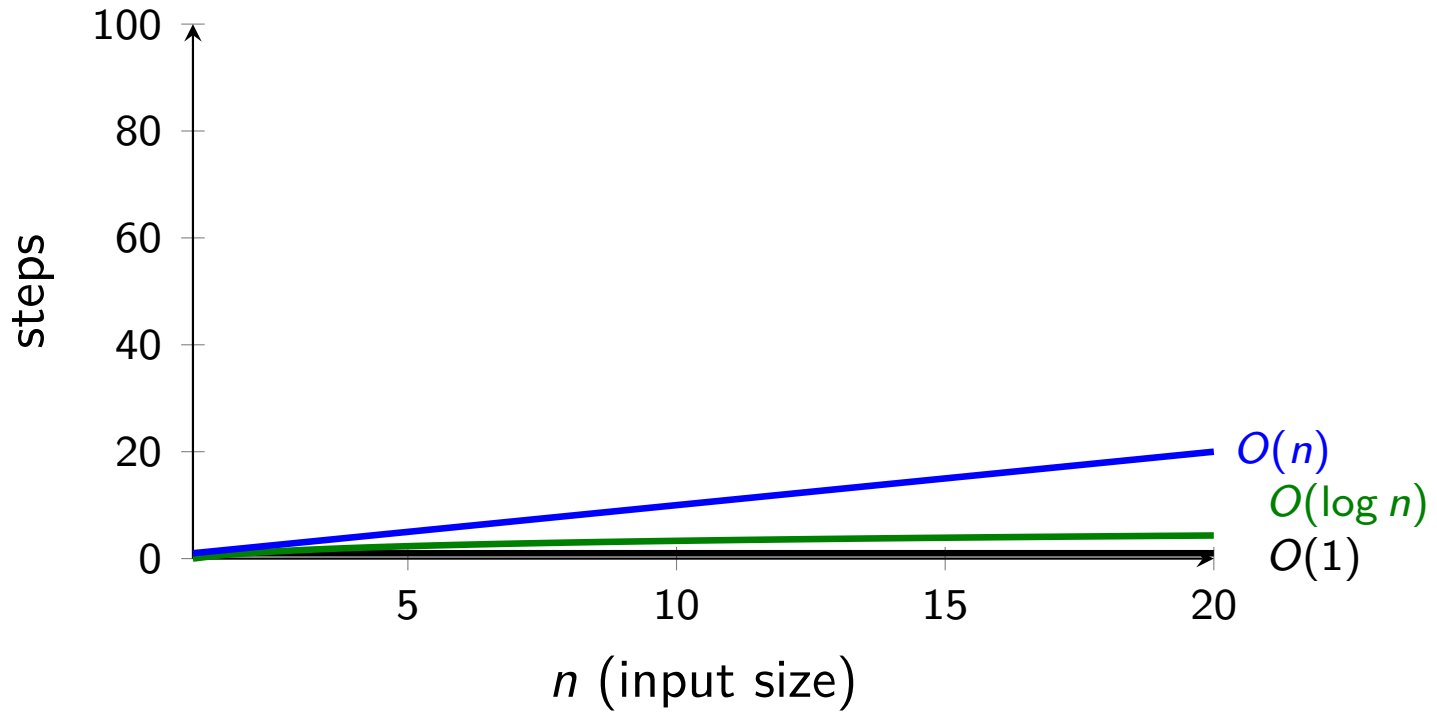
The same five growth rates drawn as functions of $n$ (number of steps on the vertical axis):




$\log_2 n$: almost flat. Doubling $n$ adds one step.

# Growth Rate Curves

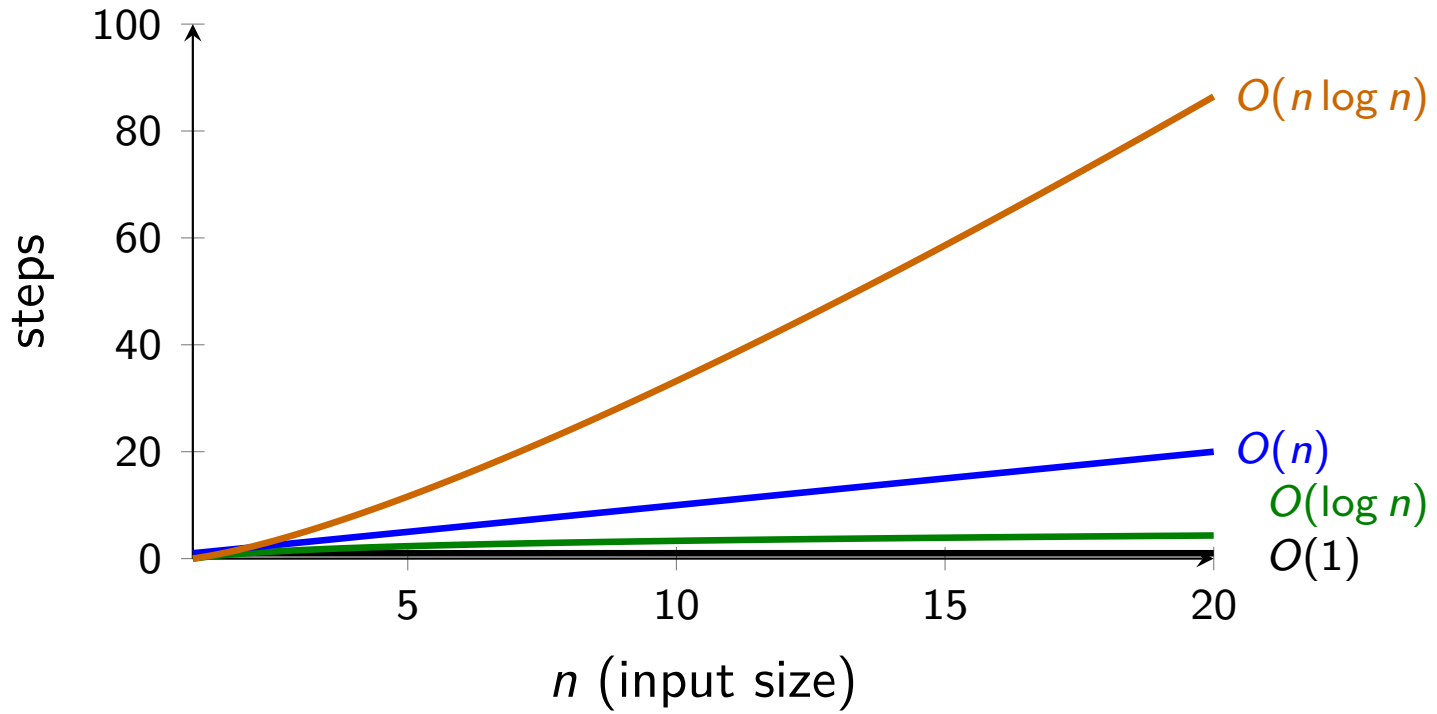
The same five growth rates drawn as functions of $n$ (number of steps on the vertical axis):




$n$: a straight line. Doubling $n$ doubles the work.

# Growth Rate Curves

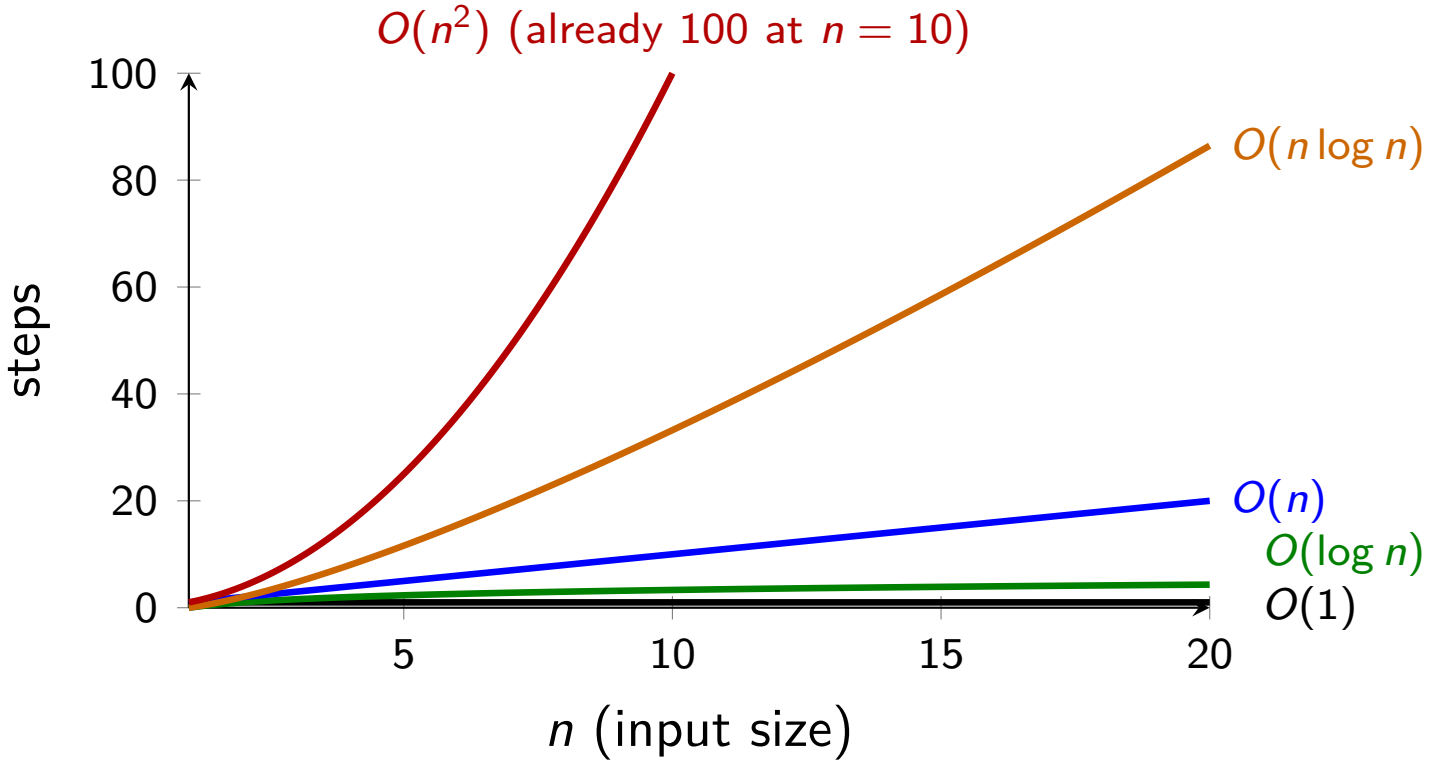
The same five growth rates drawn as functions of $n$ (number of steps on the vertical axis):




$n \log_2 n$: a little steeper than a line. Doubling $n$ slightly more than doubles the work.

# Growth Rate Curves

The same five growth rates drawn as functions of $n$ (number of steps on the vertical axis):

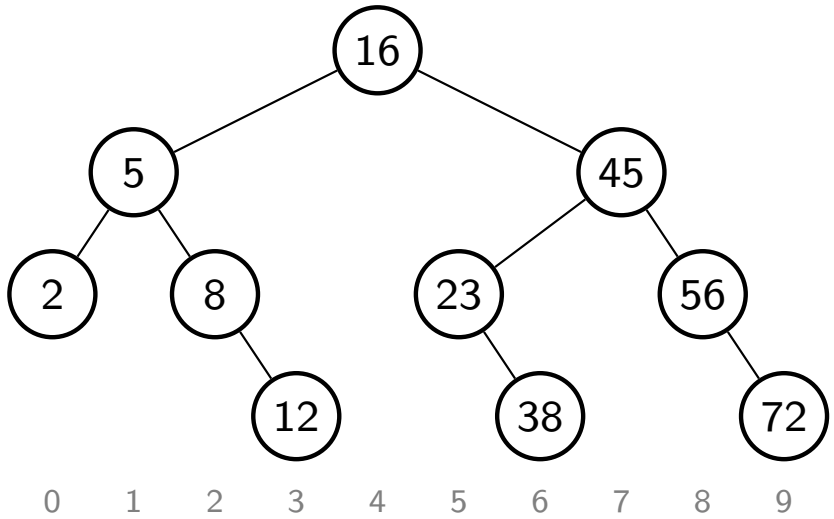



**$n^2$:** bends sharply upward. Doubling $n$ **quadruples** the work: at $n = 10$ it is already 100, at $n = 20$ it would be 400.

# Practice 1: Search for 72, Step by Step

`a = {2, 5, 8, 12, 16, 23, 38, 45, 56, 72}` (indices 0 to 9). Left: how `lo`, `hi`, `mid` change. Right: the same search as a path in the tree.

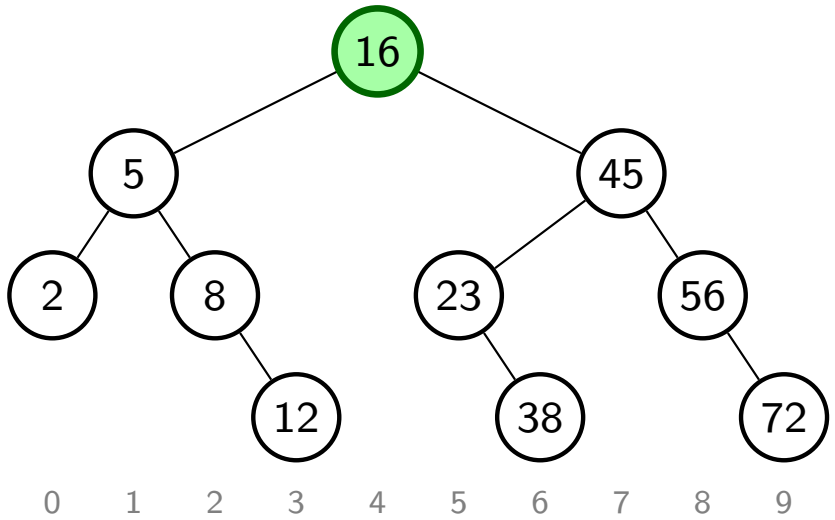
| `lo` | `hi` | `mid` | `a[mid]` | decision |
|---|---|---|---|---|
| | | | | |



# Practice 1: Search for 72, Step by Step

`a = {2, 5, 8, 12, 16, 23, 38, 45, 56, 72}` (indices 0 to 9). Left: how `lo`, `hi`, `mid` change. Right: the same search as a path in the tree.

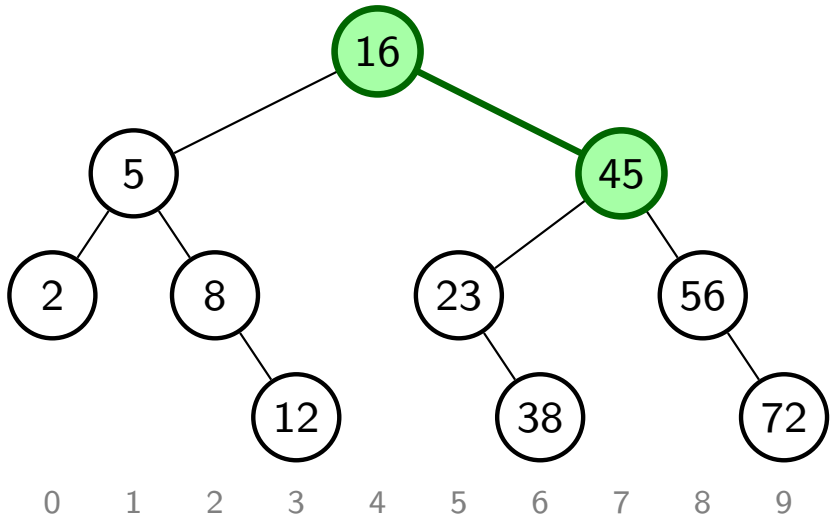
| lo | hi | mid | a[mid] | decision |
|---|---|---|---|---|
| 0 | 9 | 4 | 16 | 16 < 72: lo = 5 |



# Practice 1: Search for 72, Step by Step

`a = {2, 5, 8, 12, 16, 23, 38, 45, 56, 72}` (indices 0 to 9). Left: how `lo`, `hi`, `mid` change. Right: the same search as a path in the tree.

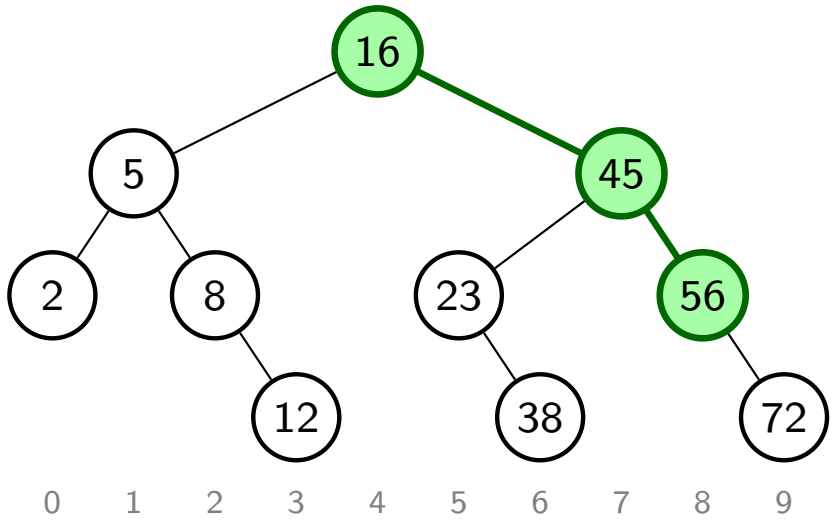
| lo | hi | mid | a[mid] | decision |
|---|---|---|---|---|
| 0 | 9 | 4 | 16 | `16 < 72: lo = 5` |
| 5 | 9 | 7 | 45 | `45 < 72: lo = 8` |



# Practice 1: Search for 72, Step by Step

`a = {2, 5, 8, 12, 16, 23, 38, 45, 56, 72}` (indices 0 to 9). Left: how `lo`, `hi`, `mid` change. Right: the same search as a path in the tree.

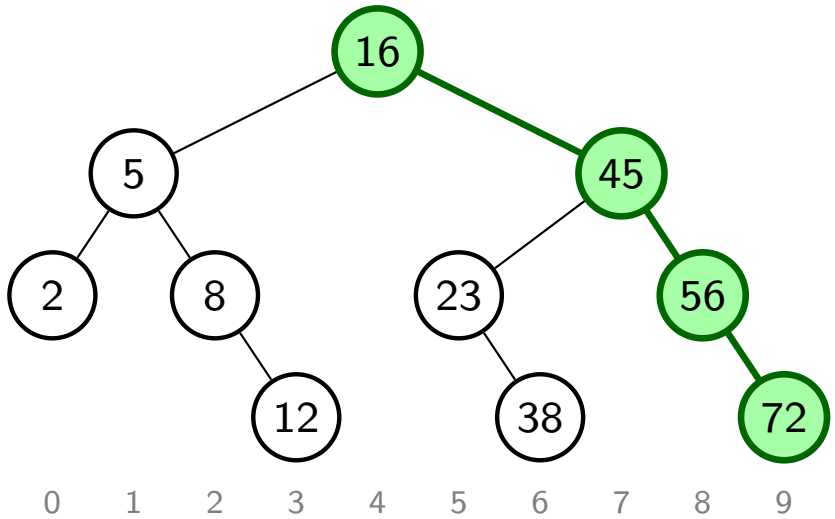
| lo | hi | mid | a[mid] | decision |
|---|---|---|---|---|
| 0 | 9 | 4 | 16 | 16 < 72: lo = 5 |
| 5 | 9 | 7 | 45 | 45 < 72: lo = 8 |
| 8 | 9 | 8 | 56 | 56 < 72: lo = 9 |



# Practice 1: Search for 72, Step by Step

`a = {2, 5, 8, 12, 16, 23, 38, 45, 56, 72}` (indices 0 to 9). Left: how `lo`, `hi`, `mid` change. Right: the same search as a path in the tree.

| `lo` | `hi` | `mid` | `a[mid]` | decision |
|---|---|---|---|---|
| 0 | 9 | 4 | 16 | `16 < 72: lo = 5` |
| 5 | 9 | 7 | 45 | `45 < 72: lo = 8` |
| 8 | 9 | 8 | 56 | `56 < 72: lo = 9` |
| 9 | 9 | 9 | 72 | **found**, return 9 |




**4 search steps.** `lo` only moves right (the target is larger every time), and each `mid` is a node on the path 16 → 45 → 56 → 72: right, right, right. Searching for the **largest** element goes down the right side of the tree.

# Practice

**1.** On `{2, 5, 8, 12, 16, 23, 38, 45, 56, 72}` search for **72**. List `mid` at each step.
**2.** How many search steps are needed (worst case) for $n = 1\,000$?
**3.** Why must the array be sorted *in the same order* as the comparison (ascending for <)?
**4.** For this binary search (worst-case number of elements examined), if $n$ grows from $10^6$ to $10^9$, how many more steps?
**5.** Binary search on a linked list: why is it $O(n)$, not $O(\log n)$?

# Practice

**1.** On `{2, 5, 8, 12, 16, 23, 38, 45, 56, 72}` search for **72**. List `mid` at each step.
**2.** How many search steps are needed (worst case) for $n = 1\,000$?
**3.** Why must the array be sorted *in the same order* as the comparison (ascending for <)?
**4.** For this binary search (worst-case number of elements examined), if $n$ grows from $10^6$ to $10^9$, how many more steps?
**5.** Binary search on a linked list: why is it $O(n)$, not $O(\log n)$?

**1.** `mid` = 4, 7, 8, 9: found 72 at index 9 (next slide: `lo`, `hi`, `mid`, and the tree). Four steps.
**2.** 10.
**3.** The decision "go left if `a[mid] > target`" assumes the left side holds smaller values. On descending data the rule must be reversed.
**4.** From 20 to 30: only 10 more steps for 1000 times more data.
**5.** Reaching the middle node needs $n/2$ pointer hops.